

05장 PySide6 기본 (PySide6 Basic)

PySide6 기본 개요

지금까지 tkinter로 GUI의 기본기를 다졌습니다. 창을 띄우고, 위젯을 올리고, 사용자의 동작에 반응하는 흐름을 경험했으니, 이제 같은 일을 더 크고 본격적인 도구로 다시 해 볼 차례입니다.

이 장에서 다루는 PySide6는 수많은 상용 프로그램이 기반으로 삼는 Qt 라이브러리를 파이썬에서 그대로 쓸 수 있게 해 주는 공식 바인딩입니다. tkinter가 가볍고 빠르게 배우기 좋은 입문 도구였다면, PySide6는 풍부한 위젯과 정교한 화면 구성, 세련된 외관까지 갖춘 실전용 도구입니다.

처음 PySide6를 접하면 클래스 이름이 길고 코드가 다소 장황하게 느껴질 수 있습니다. 하지만 그 안에는 일관된 규칙이 있습니다. 거의 모든 위젯이 하나의 부모 클래스에서 갈라져 나와 공통된 사용법을 공유하고, 사용자의 모든 동작은 시그널과 슬롯이라는 단 하나의 방식으로 처리됩니다.

이 두 가지 '위젯의 계층 구조'와 '시그널/슬롯'만 익히면, 처음 보는 위젯이라도 사용법을 미루어 짐작할 수 있게 됩니다. PySide6 학습의 절반은 사실 이 공통 원리를 체득하는 과정입니다.

이 장은 그 원리를 위젯 하나하나에 적용해 보며 익히는 데 초점을 둡니다. 먼저 모든 프로그램의 뼈대가 되는 QApplication과 이벤트 루프로 출발해, 위젯이 어떻게 부모-자식 관계로 엮이는지를 살펴봅니다. 이어서 자주 사용되는 위젯을 차례로 만나 봅니다.

목차

- [05-01 QApplication과 이벤트 루프](#)
- [05-02 위젯의 개념과 계층 구조](#)
- [05-03 라벨, 버튼, 입력창](#)
- [05-04 체크박스, 라디오버튼, 콤보박스](#)
- [05-05 슬라이더, 스펀박스, 프로그레스바](#)
- [05-06 텍스트 편집기](#)
- [05-07 레이아웃 관리](#)
- [05-08 시그널과 슬롯](#)
- [05-09 메뉴, 툴바, 상태바](#)
- [05-10 대화상자](#)

QApplication과 이벤트 루프

PySide6 프로그램은 두 가지 핵심 요소에서 시작합니다.

하나는 애플리케이션 전체를 관리하는 **QApplication**, 다른 하나는 사용자 입력을 기다리고 처리하는 **이벤트 루프**입니다.

이 둘은 화면에 직접 보이지 않지만, 없으면 창을 제대로 만들거나 유지할 수 없습니다.

- [가장 작은 PySide6 프로그램](#)
 - [첫 코드](#)
 - [코드 한 줄씩 살펴보기](#)
- [QApplication이란](#)
 - [역할](#)
 - [반드시 가장 먼저, 단 하나만](#)
 - [sys.argv는 무엇인가](#)
- [이벤트 루프의 정체](#)
 - [이벤트 루프란](#)
 - [app.exec\(\)가 하는 일](#)
 - [sys.exit\(\)로 감싸는 이유](#)
- [창 기본 설정](#)
 - [제목과 크기, 배경색을 지정한 창](#)
 - [주요 메서드](#)

가장 작은 PySide6 프로그램

첫 코드

- 빈 창 하나를 띄우는 가장 기본적인 코드입니다

```
import sys
from PySide6.QtWidgets import QApplication, QWidget

app = QApplication(sys.argv)
window = QWidget()
window.show()
sys.exit(app.exec())
```

이 6줄 안에 PySide6 프로그램의 모든 뼈대가 들어 있습니다.



코드 한 줄씩 살펴보기

- 애플리케이션 객체 생성

```
app = QApplication(sys.argv)
```

- 창(위젯) 생성

```
window = QWidget()
```

QWidget 은 버튼, 입력창, 라벨, 창 같은 GUI 요소들의 기본이 되는 클래스입니다. 여기서는 아무 내용이 없는 기본 창으로 사용합니다.

- 창을 화면에 표시

```
window.show()
```

- 이벤트 루프 시작

```
sys.exit(app.exec())
```

- `app.exec()` 가 호출되면 이벤트 루프가 시작되어 사용자 입력과 시스템 이벤트를 처리합니다.
- `sys.exit()` 는 `app.exec()` 가 반환한 종료 코드를 운영체제에 전달합니다.

QApplication이란

역할

- `QApplication` 은 GUI 프로그램 전체를 총괄하는 단 하나의 관리자입니다.
- 이벤트 루프 운영, 화면·폰트·색상 등 전역 설정 관리, 마우스·키보드 입력의 전달 같은 모든 밑바탕 작업을 담당합니다.

반드시 가장 먼저, 단 하나만

- `QApplication` 은 어떤 위젯보다 먼저 생성해야 합니다. 위젯은 내부적으로 이 객체에 의존하기 때문입니다.
- 한 프로그램에 `QApplication` 은 하나만 존재해야 합니다.

sys.argv는 무엇인가

- `sys.argv` 는 프로그램을 실행할 때 함께 넘어온 **명령행 인자 목록**입니다.
- `QApplication` 에 이를 전달하면 Qt가 인식하는 몇몇 옵션(예 : 스타일 지정)을 명령행에서 받을 수 있습니다.
- 인자를 쓸 일이 없다면 빈 리스트 `QApplication([])` 를 넘겨도 동작하지만, 관례적으로 `sys.argv` 를 넘깁니다.

이벤트 루프의 정체

이벤트 루프란

- 이벤트 루프는 "사용자가 무언가 할 때까지 끝없이 기다리는 반복문" 입니다.
- 마우스 클릭, 키 입력, 창 이동, 닫기 버튼 누르기 같은 모든 동작은 **이벤트**라는 신호로 만들어져 대기열에 쌓입니다.
- 이벤트 루프는 이 대기열을 계속 확인하면서, 이벤트가 들어오면 적절한 위젯에게 전달해 처리하게 합니다.
- 개념적으로는 다음과 같은 무한 반복과 비슷합니다.

```
while True:
    event = wait_for_event()
    if event == 종료_요청:
        break
    dispatch(event)
```

app.exec()가 하는 일

- `app.exec()` 를 호출하는 순간 이 반복문이 시작되고, 프로그램은 이벤트 루프 안에 머무르며 사용자 입력과 시스템 이벤트를 처리합니다.
- 그래서 `app.exec()` 다음 줄의 코드는 창이 닫히기 전까지 실행되지 않습니다.
- 창이 닫혀 이벤트 루프가 끝나면, `app.exec()` 는 종료 코드(보통 0)를 반환합니다.

sys.exit()로 감싸는 이유

- `sys.exit(app.exec())` 는 이벤트 루프가 반환한 종료 코드를 그대로 운영체제에 전달합니다.
- 이렇게 하면 프로그램이 정상 종료(0)인지 오류 종료인지 외부에서 구분할 수 있고, 자원도 깔끔하게 정리됩니다.
- `sys.exit` 없이 `app.exec()` 만 써도 대개 동작하지만, 권장하는 마무리 방식은 위와 같습니다.

창 기본 설정

제목과 크기, 배경색을 지정한 창

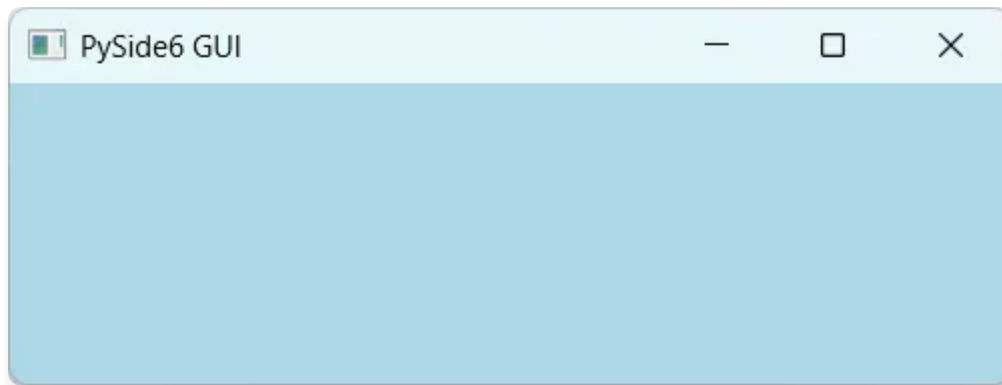
- 빈 창에 제목과 크기를 주고, 코드를 정리한 형태입니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget

app = QApplication(sys.argv)

window = QWidget()
window.setWindowTitle("PySide6 GUI")
window.resize(400, 120)
window.setStyleSheet("background-color: lightblue")

window.show()
sys.exit(app.exec())
```



주요 메서드

메서드	설명
<code>setWindowTitle(text)</code>	제목 표시줄에 표시할 텍스트 지정
<code>resize(width, height)</code>	창의 초기 크기(픽셀) 지정
<code>move(x, y)</code>	창이 뜰 화면상의 위치 지정
<code>setFixedSize(w, h)</code>	크기를 고정해 사용자가 못 바꾸게 함
<code>setStyleSheet(style)</code>	위젯의 스타일(배경색, 폰트 등)을 CSS 형식으로 지정
<code>show()</code>	창을 화면에 표시
<code>close()</code>	창을 닫음

위젯 개념과 계층 구조

PySide6에서 버튼, 글자, 입력칸처럼 화면에 보이는 요소뿐 아니라, 그것들을 담는 창도 모두 위젯입니다.

위젯은 혼자 떨어져 동작하지 않습니다. 대부분의 위젯은 부모-자식 관계로 연결되어 하나의 계층 구조를 이룹니다. 이 구조를 이해하면 위젯이 화면 어디에 표시되는지, 부모 창을 닫을 때 자식 위젯이 어떻게 정리되는지 쉽게 이해할 수 있습니다.

- [위젯이란](#)
 - [화면을 이루는 기본 단위](#)
 - [모든 위젯의 뿌리, QWidget](#)
- [부모-자식 관계](#)
 - [부모와 자식](#)
 - [화면 표시](#)
- [위젯 계층 구조](#)
 - [트리 구조](#)
 - [계층의 역할](#)
- [위젯 다뤄 보기](#)
 - [여러 자식 위젯](#)
 - [관계 확인 메서드](#)
 - [관계 확인 예제](#)

위젯이란

화면을 이루는 기본 단위

- 위젯(Widget)은 GUI 화면을 구성하는 **기본 부품**입니다.
- 버튼, 라벨, 입력창, 체크박스처럼 사용자가 보고 누르는 모든 요소가 위젯입니다.
- 이 부품들을 담는 창(Window)이나 영역(Frame)도 똑같이 위젯입니다.

모든 위젯의 뿌리, QWidget

- PySide6의 많은 위젯은 `QWidget` 을 기반으로 만들어집니다.
- 그래서 크기 변경, 위치 이동, 표시와 숨김 같은 공통 기능을 비슷한 방식으로 사용할 수 있습니다.

```
from PySide6.QtWidgets import QWidget, QPushButton, QLabel

button = QPushButton("확인")
label = QLabel("안녕하세요")
button.resize(100, 40)
label.move(10, 10)
```

부모-자식 관계

부모와 자식

- 위젯은 다른 위젯을 **자식**으로 가질 수 있습니다.
- 창 안에 버튼을 넣으면, 그 창은 **부모(parent)**, 버튼은 **자식(child)** 이 됩니다.
- 부모를 지정하는 방법은 두 가지입니다. 생성할 때 인자로 넘기거나, `setParent()` 로 나중에 지정합니다.

화면 표시

- 자식 위젯은 **부모를 통해서만** 화면에 표시됩니다.
- 부모 창을 `show()` 하면 그 안의 자식들도 함께 나타납니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QPushButton

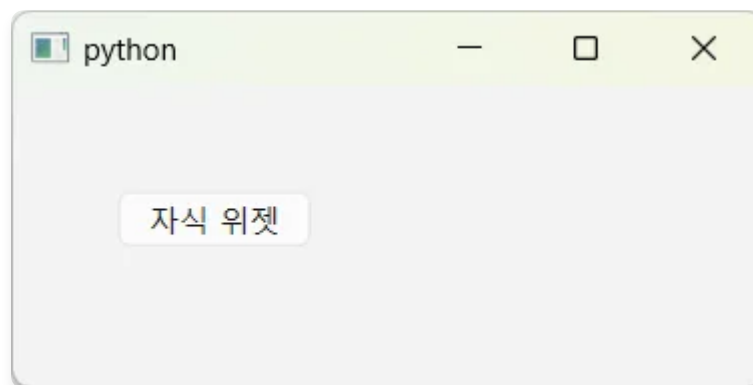
app = QApplication(sys.argv)

window = QWidget()
window.resize(300, 200)

button = QPushButton("자식 위젯", window)
button.move(80, 80)

window.show()
sys.exit(app.exec())
```

- 위 코드에서 `button` 은 `window` 의 자식이 됩니다.
- 자식 위젯은 부모 위젯 **안에** 그려지며, 부모의 좌표를 기준으로 위치가 정해집니다.



위젯 계층 구조

트리 구조

- 부모-자식 관계가 여러 단계로 이어지면 전체 위젯은 **트리(tree)** 구조를 이룹니다.
- 맨 위에 부모 없는 최상위 창이 있고, 그 아래로 자식 위젯들이, 또 그 자식의 자식들이 가지처럼 뻗어 나갑니다.

QWidget (최상위 창)

```
├─ QLabel      "제목"
├─ QPushButton "확인"
└─ QWidget     (영역을 묶는 컨테이너)
    ├─ QLineEdit 입력창
    └─ QPushButton "전송"
```

계층의 역할

- 첫째, **표시 상태가 함께 움직입니다.** 부모 위젯을 숨기면 그 안의 자식 위젯도 함께 보이지 않습니다.
- 둘째, **좌표 기준이 정해집니다.** 자식 위젯의 위치는 부모 위젯을 기준으로 계산됩니다.
- 셋째, **삭제가 함께 처리됩니다.** 부모 위젯이 삭제되면 그 아래에 있는 자식 위젯들도 함께 정리됩니다.

위젯 다뤄 보기

여러 자식 위젯

- 창 하나에 라벨과 버튼을 함께 올려, 부모-자식 관계를 직접 확인하는 예제입니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QLabel, QPushButton

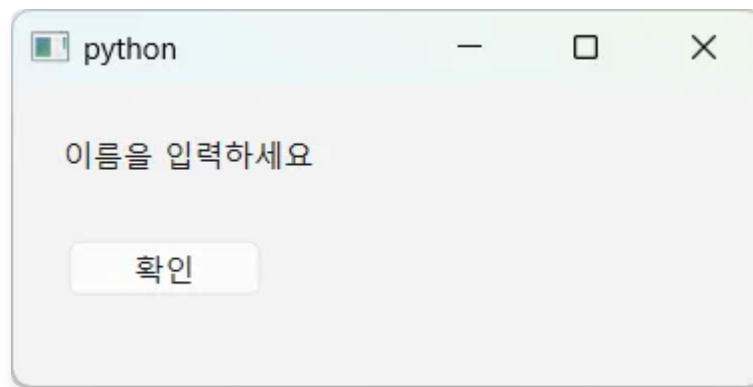
app = QApplication(sys.argv)

window = QWidget()
window.resize(300, 120)

label = QLabel("이름을 입력하세요", window)
label.move(20, 20)

button = QPushButton("확인", window)
button.move(20, 60)

window.show()
sys.exit(app.exec())
```



관계 확인 메서드

- 부모-자식 관계는 다음 메서드로 확인할 수 있습니다.

메서드	설명
<code>parent()</code>	이 위젯의 부모 위젯을 반환 (없으면 None)
<code>children()</code>	이 위젯이 가진 자식 위젯 목록을 반환
<code>setParent(widget)</code>	부모를 나중에 지정하거나 변경
<code>show()</code> / <code>hide()</code>	위젯(과 자식들)을 표시하거나 숨김
<code>isVisible()</code>	현재 화면에 보이는 상태인지 확인
<code>deleteLater()</code>	위젯(과 자식들)을 안전하게 삭제 예약

관계 확인 예제

- `parent()` 와 `children()` 으로 위젯 트리를 직접 확인할 수 있습니다.
- 위 코드에 아래 두 줄을 추가합니다.

```
print(button.parent())
print(window.children())
```

라벨, 버튼, 입력창

대부분의 프로그램은 무언가를 보여주고(라벨), 사용자가 누르고(버튼), 값을 입력받는(입력창) 세 가지 동작을 기본으로 합니다. PySide6에서는 각각 QLabel, QPushButton, QLineEdit 가 이 역할을 맡습니다.

이 장에서는 이 세 위젯의 사용법과 함께, 버튼을 눌렀을 때 동작을 연결하는 **시그널과 슬롯**의 첫걸음을 익힙니다.

- [라벨](#)
 - [글자를 보여주는 위젯](#)
 - [텍스트 다루기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
 - [주요 시그널](#)
- [버튼](#)
 - [누르는 위젯](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
 - [주요 시그널](#)
- [시그널과 슬롯](#)
 - [이벤트 처리 구조](#)
 - [동작 흐름](#)
- [입력창](#)
 - [한 줄을 입력받는 위젯](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
 - [주요 시그널](#)
- [세 위젯 함께 쓰기](#)
 - [입력값을 라벨에 표시하는 예제](#)

라벨

글자를 보여주는 위젯

- `QLabel` 은 화면에 텍스트나 이미지를 표시하는 가장 단순한 위젯입니다.

```
from PySide6.QtWidgets import QApplication, QWidget, QLabel

label = QLabel("안녕하세요", window)
```

텍스트 다루기

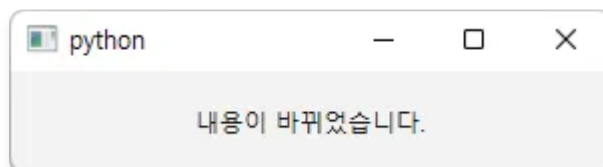
- 라벨 생성 후 `setText()` 로 내용을 변경할 수 있습니다.
- 글자의 정렬은 `setAlignment()` 로 지정합니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QLabel
from PySide6.QtCore import Qt

app = QApplication(sys.argv)
window = QWidget()
window.resize(300,50)

label = QLabel("안녕하세요", window)
label.setText("내용이 바뀌었습니다.")
label.setAlignment(Qt.AlignmentFlag.AlignCenter)
label.resize(window.size())

window.show()
sys.exit(app.exec())
```



주요 메서드

메서드	설명
<code>setText(str)</code>	라벨에 표시할 텍스트 설정
<code>text()</code>	현재 텍스트 반환
<code>setWordWrap(bool)</code>	자동 줄바꿈 여부 설정
<code>setStyleSheet(str)</code>	CSS 스타일 적용
<code>setTextFormat(Qt.TextFormat)</code>	텍스트 포맷 지정 (Plain/Rich/Markdown)
<code>setAlignment(Qt.Alignment)</code>	정렬 방식 설정
<code>clear()</code>	내용 비우기

주요 옵션

옵션	허용 값	설명
<code>alignment</code>	<code>Qt.AlignLeft</code> , <code>Qt.AlignRight</code> , <code>Qt.AlignCenter</code> 등	정렬
<code>wordWrap</code>	<code>True</code> / <code>False</code>	자동 줄바꿈
<code>scaledContents</code>	<code>True</code> / <code>False</code>	이미지 크기 자동 조정
<code>indent</code>	정수	들여쓰기 픽셀
<code>margin</code>	정수	내부 여백 픽셀
<code>openExternalLinks</code>	<code>True</code> / <code>False</code>	외부 링크 자동 열기

주요 시그널

시그널	설명
<code>linkActivated(str)</code>	라벨 내 링크가 클릭되었을 때 발생
<code>linkHovered(str)</code>	라벨 내 링크 위에 마우스가 올라갔을 때 발생

버튼

누르는 위젯

- `QPushButton` 은 사용자의 클릭 이벤트를 처리하는 위젯입니다. 생성 후 반드시 슬롯을 연결해야 제 역할을 합니다.

```
from PySide6.QtWidgets import QPushButton

button = QPushButton("확인", window)
```

주요 메서드

메서드	설명
<code>setText(str)</code>	버튼에 표시할 텍스트 설정
<code>setStyleSheet(str)</code>	CSS 스타일 적용
<code>setEnabled(bool)</code>	활성/비활성 상태 설정
<code>setCheckable(bool)</code>	토글(체크) 가능 여부 설정
<code>isChecked()</code>	체크(눌림) 상태 반환
<code>setDefault(bool)</code>	기본 버튼 지정 (Enter로 동작)
<code>text()</code>	현재 텍스트 반환

주요 옵션

옵션	허용 값	설명
<code>enabled</code>	<code>True</code> / <code>False</code>	클릭 가능 여부
<code>checkable</code>	<code>True</code> / <code>False</code>	토글 버튼으로 동작 여부
<code>checked</code>	<code>True</code> / <code>False</code>	눌림 상태
<code>default</code>	<code>True</code> / <code>False</code>	기본 버튼 여부
<code>flat</code>	<code>True</code> / <code>False</code>	평면(테두리 없음) 스타일

주요 시그널

시그널	설명
<code>clicked(bool)</code>	버튼이 클릭되었을 때 발생 (가장 많이 사용)
<code>pressed()</code>	버튼을 누르는 순간 발생
<code>released()</code>	버튼에서 손을 뗄 때 발생
<code>toggled(bool)</code>	체크 상태가 바뀔 때 발생 (checkable일 때)

시그널과 슬롯

- PySide6의 핵심 이벤트 처리 방식입니다. "어떤 일이 발생하면(시그널), 이 함수를 실행해라(슬롯)" 는 규칙입니다.

이벤트 처리 구조

- PySide6의 위젯에서 발생하는 이벤트를 **시그널(signal)** 이라 합니다. 버튼이 눌리면 `clicked` 시그널이 발생합니다.
- 이벤트 발생 시 실행할 일반 함수를 **슬롯(slot)** 이라 합니다.
- `위젯.시그널.connect(슬롯)` 형태로 둘을 이어 주면, 신호가 발생할 때마다 슬롯이 자동으로 실행됩니다.

```
def on_click():
    print("버튼이 눌렸습니다")

button = QPushButton("클릭", window)
button.clicked.connect(on_click)
```

동작 흐름

- 사용자가 버튼을 누름 → 버튼이 `clicked` 시그널 발생 → 연결된 `on_click()` 이 실행됩니다.
- 이 구조 덕분에 "무엇이 일어났는가"와 "그래서 무엇을 할 것인가"를 깔끔하게 나눠 작성할 수 있습니다.

⚠ 여기서서는 시그널과 슬롯을 간략히 살펴보았습니다. 이와 관련된 보다 상세한 내용은 별도의 장을 할애하여 다루고 있습니다.

입력창

한 줄을 입력받는 위젯

- `QLineEdit` 은 사용자로부터 한 줄의 텍스트를 입력받는 위젯입니다.

```
from PySide6.QtWidgets import QLineEdit

edit = QLineEdit(window)
edit.setPlaceholderText("이름을 입력하세요")
```

주요 메서드

메서드	설명
<code>text()</code>	현재 입력된 텍스트 반환
<code>setText(str)</code>	텍스트 설정
<code>clear()</code>	텍스트 전체 삭제
<code>setPlaceholderText(str)</code>	비어 있을 때 표시할 안내 문구 설정
<code>setEchoMode(mode)</code>	표시 방식 설정 (Normal, Password 등)
<code>setReadOnly(bool)</code>	읽기 전용 여부 설정
<code>setMaxLength(int)</code>	최대 입력 길이 설정
<code>setValidator(validator)</code>	입력 검증기 설정(숫자만, 이메일 형식만 등)

주요 옵션

옵션	허용 값	설명
<code>text</code>	문자열	라인 에디트에 입력된 텍스트
<code>placeholderText</code>	문자열	텍스트가 비어 있을 때 열게 표시되는 안내 문구
<code>maxLength</code>	정수	사용자가 입력할 수 있는 최대 글자 수
<code>echoMode</code>	<code>QLineEdit.EchoMode</code> (<code>Normal</code> , <code>Password</code> 등)	텍스트 표시 방식

주요 시그널

시그널	설명
<code>textChanged(str)</code>	텍스트가 변경될 때 발생 (프로그램적 변경 포함)
<code>textEdited(str)</code>	사용자가 직접 편집할 때만 발생
<code>returnPressed()</code>	Enter 키 입력 시 발생
<code>editingFinished()</code>	편집 완료(포커스 이탈 또는 Enter) 시 발생

세 위젯 함께 쓰기

입력값을 라벨에 표시하는 예제

- 입력창에 문자를 입력하고 버튼을 누르면, 라벨에 문자가 출력되는 프로그램입니다.
- 라벨, 버튼, 입력창과 시그널/슬롯 연결을 확인할 수 있습니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QLabel, QPushButton, QLineEdit

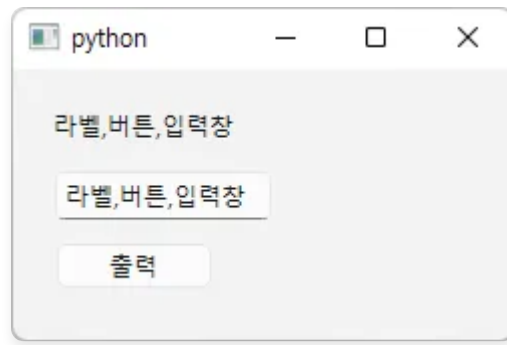
def on_click():
    label.setText(edit.text())
    label.adjustSize()

app = QApplication(sys.argv)
window = QWidget()

label = QLabel("결과", window)
label.move(20, 20)
edit = QLineEdit(window)
edit.move(20, 50)
button = QPushButton("출력", window)
button.move(20, 85)

button.clicked.connect(on_click)

window.show()
sys.exit(app.exec())
```



체크박스, 라디오, 콤보박스

이장에서는 사용자에게 선택지를 제시하고 고르게 하는 위젯을 다룹니다. PySide6에서는 QCheckBox, QRadioButton, QComboBox 가 각각 여러 항목 중복 선택, 하나만 선택, 목록에서 고르기 역할을 맡습니다.

이 장에서는 세 위젯의 사용법을 익힙니다.

- [체크박스](#)
 - [켜고 끄는 위젯](#)
 - [상태 다루기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
 - [주요 시그널](#)
- [라디오버튼](#)
 - [하나만 고르는 위젯](#)
 - [그룹으로 묶기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
 - [주요 시그널](#)
- [콤보박스](#)
 - [목록에서 고르는 위젯](#)
 - [항목 다루기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
 - [주요 시그널](#)

체크박스

켜고 끄는 위젯

- `QCheckBox` 는 선택/해제 두 상태를 가지는 위젯입니다. 여러 개를 동시에 켤 수 있어 다중 선택에 적합합니다.

```
from PySide6.QtWidgets import QApplication, QWidget, QCheckBox

check = QCheckBox("앱 푸시 동의", window)
```

상태 다루기

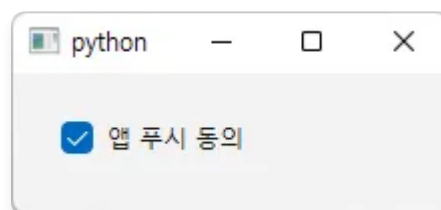
- 생성 후 `setChecked()` 로 초기 상태를 지정할 수 있습니다.
- 현재 선택 여부는 `isChecked()` 로 확인합니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QCheckBox

app = QApplication(sys.argv)
window = QWidget()

check = QCheckBox("앱 푸시 동의", window)
check.setChecked(True)
check.move(20, 20)

window.show()
sys.exit(app.exec())
```



주요 메서드

메서드	설명
<code>setText(str)</code>	체크박스 옆 텍스트 설정
<code>text()</code>	현재 텍스트 반환
<code>setChecked(bool)</code>	선택 상태 설정
<code>isChecked()</code>	선택 여부 반환
<code>setTristate(bool)</code>	3단계(선택/해제/중간) 상태 사용 여부
<code>checkState()</code>	현재 상태 반환 (Unchecked/PartiallyChecked/Checked)
<code>setStyleSheet(str)</code>	CSS 스타일 적용

주요 옵션

옵션	허용 값	설명
<code>checked</code>	<code>True</code> / <code>False</code>	선택 상태
<code>text</code>	문자열	표시 텍스트
<code>tristate</code>	<code>True</code> / <code>False</code>	3단계 상태 사용 여부
<code>enabled</code>	<code>True</code> / <code>False</code>	클릭 가능 여부

주요 시그널

시그널	설명
<code>stateChanged(int)</code>	상태가 바뀔 때 발생 (0/1/2 값 전달)
<code>toggled(bool)</code>	선택 상태가 바뀔 때 발생 (가장 많이 사용)
<code>clicked(bool)</code>	체크박스가 클릭되었을 때 발생

라디오버튼

하나만 고르는 위젯

- `QRadioButton` 은 여러 항목 중 하나만 선택하게 하는 위젯입니다. 같은 부모(또는 그룹) 안에서는 하나를 켜면 나머지가 자동으로 꺼집니다.

```
from PySide6.QtWidgets import QRadioButton

radio = QRadioButton("남성", window)
```

그룹으로 묶기

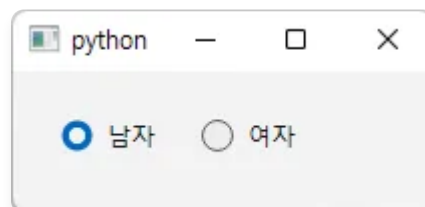
- 같은 부모 위젯에 배치하면 자동으로 한 그룹이 되어 상호 배타적으로 동작합니다.
- 여러 그룹을 명시적으로 나누려면 `QButtonGroup` 을 사용합니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QRadioButton

app = QApplication(sys.argv)
window = QWidget()

radio1 = QRadioButton("남성", window)
radio1.move(20, 20)
radio2 = QRadioButton("여성", window)
radio2.move(50, 20)
radio1.setChecked(True)

window.show()
sys.exit(app.exec())
```



주요 메서드

메서드	설명
<code>setText(str)</code>	라디오버튼 옆 텍스트 설정
<code>text()</code>	현재 텍스트 반환
<code>setChecked(bool)</code>	선택 상태 설정
<code>isChecked()</code>	선택 여부 반환
<code>setStyleSheet(str)</code>	CSS 스타일 적용

주요 옵션

옵션	허용 값	설명
<code>checked</code>	<code>True</code> / <code>False</code>	선택 상태
<code>text</code>	문자열	표시 텍스트
<code>autoExclusive</code>	<code>True</code> / <code>False</code>	같은 그룹 내 자동 배타 선택 여부
<code>enabled</code>	<code>True</code> / <code>False</code>	클릭 가능 여부

주요 시그널

시그널	설명
<code>toggled(bool)</code>	선택 상태가 바뀔 때 발생 (가장 많이 사용)
<code>clicked(bool)</code>	라디오버튼이 클릭되었을 때 발생
<code>pressed()</code>	누르는 순간 발생

콤보박스

목록에서 고르는 위젯

- `QComboBox` 는 드롭다운 목록에서 한 항목을 선택하는 위젯입니다. 항목이 많을 때 공간을 절약할 수 있습니다.

```
from PySide6.QtWidgets import QComboBox

combo = QComboBox(window)
combo.addItems(["대중교통", "자가용", "도보"])
```

항목 다루기

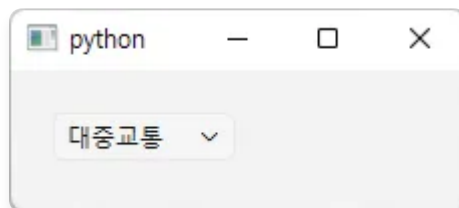
- `addItem()` 또는 `addItems()` 로 항목을 추가합니다.
- 현재 선택된 값은 `currentText()`, 인덱스는 `currentIndex()` 로 얻습니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QComboBox

app = QApplication(sys.argv)
window = QWidget()

combo = QComboBox(window)
combo.addItems(["대중교통", "자가용", "도보"])
combo.setCurrentIndex(1)
combo.move(20, 20)

window.show()
sys.exit(app.exec())
```



주요 메서드

메서드	설명
<code>addItem(str)</code>	항목 하나 추가
<code>addItems(list)</code>	여러 항목 한 번에 추가
<code>insertItem(int, str)</code>	특정 위치에 항목 삽입
<code>removeItem(int)</code>	특정 항목 삭제
<code>clear()</code>	모든 항목 삭제
<code>currentText()</code>	현재 선택된 텍스트 반환
<code>currentIndex()</code>	현재 선택된 인덱스 반환
<code>setCurrentIndex(int)</code>	선택 항목을 인덱스로 지정
<code>count()</code>	전체 항목 수 반환
<code>setEditable(bool)</code>	직접 입력 가능 여부 설정

주요 옵션

옵션	허용 값	설명
<code>currentIndex</code>	정수	현재 선택된 항목 인덱스
<code>currentText</code>	문자열	현재 선택된 항목 텍스트
<code>editable</code>	<code>True</code> / <code>False</code>	직접 입력 가능 여부
<code>maxVisibleItems</code>	정수	드롭다운에 한 번에 보일 항목 수
<code>enabled</code>	<code>True</code> / <code>False</code>	선택 가능 여부

주요 시그널

시그널	설명
<code>currentIndexChanged(int)</code>	선택 인덱스가 바뀔 때 발생
<code>currentTextChanged(str)</code>	선택 텍스트가 바뀔 때 발생 (가장 많이 사용)
<code>activated(int)</code>	사용자가 항목을 직접 선택했을 때 발생
<code>editTextChanged(str)</code>	(editable일 때) 입력 텍스트가 변경될 때 발생

슬라이더와 프로그레스바

이 장에서는 사용자에게 수치 값을 입력받거나 진행 상태를 보여주는 위젯을 다룹니다. PySide6에서는 QSlider, QSpinBox, QProgressBar가 각각 막대를 끌어 값 조절, 화살표나 입력으로 값 조절, 작업 진행률 표시 역할을 맡습니다.

이 장에서는 세 위젯의 사용법을 익힙니다.

- [슬라이더](#)
 - [끌어서 값을 정하는 위젯](#)
 - [범위와 값 다루기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
 - [주요 시그널](#)
- [스핀박스](#)
 - [화살표로 값을 조절하는 위젯](#)
 - [범위와 단위 다루기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
 - [주요 시그널](#)
- [프로그레스바](#)
 - [진행 상태를 보여주는 위젯](#)
 - [진행률 다루기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
 - [주요 시그널](#)

슬라이더

끌어서 값을 정하는 위젯

- `QSlider` 는 막대를 좌우(또는 상하)로 끌어 값을 조절하는 위젯입니다. 음량, 밝기처럼 연속적인 범위의 값을 직관적으로 입력받을 때 적합합니다.

```
from PySide6.QtCore import Qt
from PySide6.QtWidgets import QApplication, QWidget, QSlider

slider = QSlider(Qt.Horizontal, window)
```

범위와 값 다루기

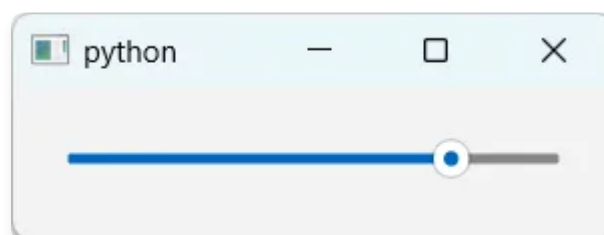
- `setRange()` 또는 `setMinimum()` / `setMaximum()` 으로 값의 범위를 지정합니다.
- 현재 값은 `value()` 로 얻고, `setValue()` 로 지정합니다.

```
import sys
from PySide6.QtCore import Qt
from PySide6.QtWidgets import QApplication, QWidget, QSlider

app = QApplication(sys.argv)
window = QWidget()

slider = QSlider(Qt.Horizontal, window)
slider.setRange(0, 100)
slider.setValue(30)
slider.move(20, 20)
slider.resize(200, 20)

window.show()
sys.exit(app.exec())
```



주요 메서드

메서드	설명
<code>setRange(min, max)</code>	값의 최소·최대 범위 설정
<code>setMinimum(int)</code>	최솟값 설정
<code>setMaximum(int)</code>	최댓값 설정
<code>setValue(int)</code>	현재 값 설정
<code>value()</code>	현재 값 반환
<code>setOrientation(Qt.Orientation)</code>	가로/세로 방향 설정
<code>setSingleStep(int)</code>	화살표 키로 움직이는 단위 설정
<code>setTickPosition(pos)</code>	눈금 표시 위치 설정
<code>setStyleSheet(str)</code>	CSS 스타일 적용

주요 옵션

옵션	허용 값	설명
<code>value</code>	정수	현재 값
<code>minimum</code>	정수	최솟값
<code>maximum</code>	정수	최댓값
<code>orientation</code>	<code>Qt.Horizontal</code> / <code>Qt.Vertical</code>	막대 방향
<code>singleStep</code>	정수	한 단계 이동 단위
<code>enabled</code>	<code>True</code> / <code>False</code>	조작 가능 여부

주요 시그널

시그널	설명
<code>valueChanged(int)</code>	값이 바뀔 때 발생
<code>sliderMoved(int)</code>	사용자가 막대를 끌 때 발생
<code>sliderPressed()</code>	막대를 누르는 순간 발생
<code>sliderReleased()</code>	막대에서 손을 뗄 때 발생

스핀박스

화살표로 값을 조절하는 위젯

- `QSpinBox` 는 위·아래 화살표나 직접 입력으로 정수 값을 조절하는 위젯입니다. 수량, 나이처럼 정해진 단위로 증감하는 값을 입력받을 때 적합합니다.
- 실수 값을 다룰 때는 `QDoubleSpinBox` 를 사용합니다.

```
from PySide6.QtWidgets import QSpinBox

spin = QSpinBox(window)
spin.setRange(0, 12)
```

범위와 단위 다루기

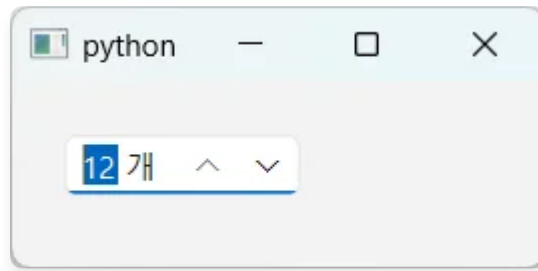
- `setRange()` 로 입력 가능한 범위를 지정합니다.
- `setPrefix()` / `setSuffix()` 로 값 앞뒤에 단위 문자열을 붙일 수 있습니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QSpinBox

app = QApplication(sys.argv)
window = QWidget()

spin = QSpinBox(window)
spin.setRange(0, 12)
spin.setSuffix(" 개 ")
spin.move(20, 20)

window.show()
sys.exit(app.exec())
```



주요 메서드

메서드	설명
<code>setRange(min, max)</code>	값의 최소·최대 범위 설정
<code>setValue(int)</code>	현재 값 설정
<code>value()</code>	현재 값 반환
<code>setSingleStep(int)</code>	화살표 클릭 시 증감 단위 설정
<code>setPrefix(str)</code>	값 앞에 붙일 문자열 설정
<code>setSuffix(str)</code>	값 뒤에 붙일 문자열 설정
<code>setWrapping(bool)</code>	최대·최소에서 값 순환 여부
<code>setStyleSheet(str)</code>	CSS 스타일 적용

주요 옵션

옵션	허용 값	설명
<code>value</code>	정수	현재 값
<code>minimum</code>	정수	최솟값
<code>maximum</code>	정수	최댓값
<code>singleStep</code>	정수	한 단계 증감 단위
<code>prefix</code>	문자열	값 앞 단위 표시
<code>suffix</code>	문자열	값 뒤 단위 표시
<code>enabled</code>	<code>True</code> / <code>False</code>	입력 가능 여부

주요 시그널

시그널	설명
<code>valueChanged(int)</code>	값이 바뀔 때 발생
<code>textChanged(str)</code>	표시 텍스트가 바뀔 때 발생
<code>editingFinished()</code>	입력을 마쳤을 때 발생

프로그레스바

진행 상태를 보여주는 위젯

- `QProgressBar` 는 작업의 진행률을 막대로 표시하는 위젯입니다. 파일 복사, 다운로드처럼 시간이 걸리는 작업의 상태를 사용자에게 알릴 때 적합합니다.

```
from PySide6.QtWidgets import QProgressBar

bar = QProgressBar(window)
bar.setValue(37)
```

진행률 다루기

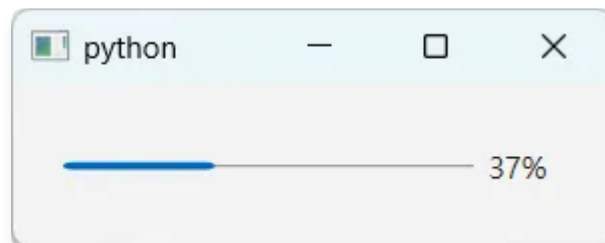
- `setRange()` 또는 `setMinimum()` / `setMaximum()` 으로 범위를 지정합니다.
- `setValue()` 로 현재 진행 값을 설정합니다.
- 최소·최대를 모두 0으로 두면 진행률을 알 수 없는 **무한(busy)** 상태로 표시됩니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QProgressBar

app = QApplication(sys.argv)
window = QWidget()

bar = QProgressBar(window)
bar.setRange(0, 100)
bar.setValue(37)
bar.move(20, 20)
bar.resize(200, 25)

window.show()
sys.exit(app.exec())
```



주요 메서드

메서드	설명
<code>setRange(min, max)</code>	진행률 범위 설정
<code>setMinimum(int)</code>	최솟값 설정
<code>setMaximum(int)</code>	최댓값 설정
<code>setValue(int)</code>	현재 진행 값 설정
<code>value()</code>	현재 진행 값 반환
<code>reset()</code>	진행 상태 초기화
<code>setTextVisible(bool)</code>	퍼센트 텍스트 표시 여부
<code>setFormat(str)</code>	표시 형식 설정 (예: <code>"%p%"</code>)
<code>setStyleSheet(str)</code>	CSS 스타일 적용

주요 옵션

옵션	허용 값	설명
<code>value</code>	정수	현재 진행 값
<code>minimum</code>	정수	최솟값
<code>maximum</code>	정수	최댓값
<code>textVisible</code>	<code>True</code> / <code>False</code>	퍼센트 텍스트 표시 여부
<code>format</code>	문자열	표시 형식
<code>orientation</code>	<code>Qt.Horizontal</code> / <code>Qt.Vertical</code>	막대 방향

주요 시그널

시그널	설명
<code>valueChanged(int)</code>	진행 값이 바뀔 때 발생

텍스트 편집기

메모, 게시글, 로그 출력처럼 여러 줄에 걸친 긴 글을 다루야 할 때가 있습니다. 이때 쓰는 것이 텍스트 편집기 위젯입니다. PySide6는 서식 있는 글을 다루는 QTextEdit와 순수한 글자만 빠르게 다루는 QPlainTextEdit를 제공합니다.

이 장에서는 두 위젯의 차이와 사용법, 그리고 이를 활용한 간단한 메모장 형태의 예제를 익힙니다.

- [QTextEdit](#)
 - [긴 글을 다루는 위젯](#)
 - [값 읽고 쓰기](#)
 - [주요 메서드](#)
- [QPlainTextEdit](#)
 - [서식 없는 순수 텍스트 전용](#)
 - [주요 메서드](#)

QTextEdit

긴 글을 다루는 위젯

- `QTextEdit` 은 여러 줄의 텍스트를 입력하고 편집하는 위젯입니다. 줄바꿈, 스크롤이 자동으로 처리됩니다.
- 글자에 굵게·색상·글꼴 같은 서식을 줄 수 있고, 간단한 HTML도 표시할 수 있습니다.

```
from PySide6.QtWidgets import QApplication, QWidget, QTextEdit

editor = QTextEdit("이 곳에 텍스트 입력합니다.")
```

값 읽고 쓰기

- `toPlainText()` 로 서식 없는 순수 글자를, `toHtml()` 로 서식이 담긴 HTML을 읽습니다.
- `setPlainText()` 로 글자를 채우고, `setHtml()` 로 서식 있는 내용을 넣습니다.

```

import sys
from PySide6.QtWidgets import QApplication, QWidget, QHBoxLayout, QTextEdit, QTextBrowser,

app = QApplication(sys.argv)
window = QWidget()
layout = QHBoxLayout()

editor = QTextEdit("이곳에 텍스트 입력합니다.")
result = QTextBrowser()

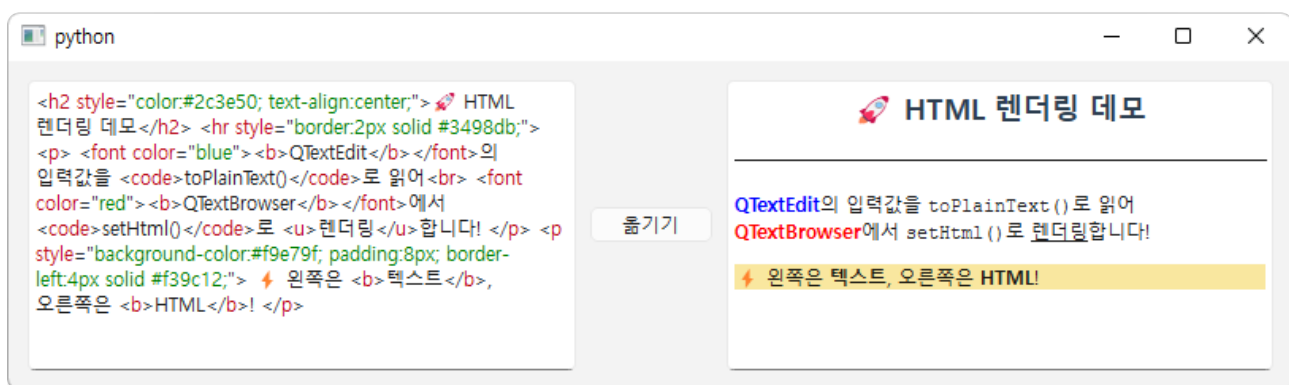
def read_write():
    result.setHtml(editor.toPlainText())

button = QPushButton("출기기")
button.clicked.connect(read_write)

layout.addWidget(editor)
layout.addWidget(button)
layout.addWidget(result)

window.setLayout(layout)
window.show()
sys.exit(app.exec())

```



주요 메서드

메서드	설명
<code>toPlainText()</code>	서식 없는 순수 텍스트 읽기
<code>setPlainText(text)</code>	순수 텍스트로 내용 채우기
<code>toHtml()</code> / <code>setHtml(...)</code>	서식(HTML) 읽기 / 넣기
<code>append(text)</code>	맨 끝에 한 줄 추가
<code>setReadOnly(True)</code>	읽기 전용으로 설정
<code>clear()</code>	내용 전체 삭제

📖 알아두기

`QTextBrowser` 는 `QTextEdit` 를 상속받은 **읽기 전용 리치 텍스트 뷰어**입니다. HTML 이나 도움말 출력에 특화되어 내용 수정은 불가하지만 다음과 같은 편의 기능이 추가되어 있습니다.

- **하이퍼링크** : 문서 내 `<a href="...">` 링크 클릭 및 외부 URL 열기가 가능합니다.
- **문서 탐색** : `backward()` , `forward()` , `home()` 메서드로 웹 브라우저처럼 앞뒤 페이지를 이동할 수 있습니다.
- **앵커 이동** : 문서 안의 특정 위치(`#anchor`)로 점프할 수 있습니다.

QPlainTextEdit

서식 없는 순수 텍스트 전용

- `QPlainTextEdit` 은 서식 기능을 덜어내고 **순수한 글자만 빠르고 가볍게** 다루는 위젯입니다.
- 로그 출력, 코드 입력, 대량의 텍스트처럼 서식이 필요 없고 **양이 많은** 경우에 적합합니다.

```
from PySide6.QtWidgets import QApplication, QPlainTextEdit

app = QApplication([])

log = QPlainTextEdit()
log.appendPlainText("프로그램을 시작했습니다")
print(log.toPlainText())
```

주요 메서드

메서드	설명
<code>toPlainText()</code>	서식 없는 순수 텍스트 읽기
<code>setPlainText(text)</code>	순수 텍스트로 내용 채우기
<code>appendPlainText(text)</code>	맨 끝에 한 단락(줄) 추가
<code>insertPlainText(text)</code>	커서 위치에 텍스트 삽입
<code>setReadOnly(True)</code>	읽기 전용으로 설정
<code>clear()</code>	내용 전체 삭제
<code>setMaximumBlockCount(n)</code>	최대 블록(줄) 수 제한 (로그 출력 등에 유용)
<code>setLineWrapMode(mode)</code>	줄 바꿈 모드 설정 (NoWrap / WidgetWidth)

알아두기

둘 중 무엇을 쓸까

글자에 색·굵기 등 서식이 필요하다면 `QTextEdit`, 서식 없이 양 많은 순수 텍스트면 `QPlainTextEdit` 을 고르면 됩니다.

레이아웃 관리

이 장에서는 위젯을 배치할 때 가장 많이 쓰는 네 가지 레이아웃 QVBoxLayout, QHBoxLayout, QGridLayout, QFormLayout을 익힙니다.

- [레이아웃이란](#)
 - [좌표 대신 규칙으로 배치](#)
 - [동일한 사용 순서](#)
 - [네 가지 주요 레이아웃](#)
- [QVBoxLayout](#)
 - [위에서 아래로 쌓기](#)
 - [주요 메서드](#)
- [HHBoxLayout](#)
 - [왼쪽에서 오른쪽으로 늘어놓기](#)
 - [스프링으로 정렬하기](#)
- [QGridLayout](#)
 - [행과 열의 표처럼 배치](#)
 - [여러 칸 합치기](#)
 - [주요 메서드](#)
- [QFormLayout](#)
 - [라벨과 입력칸의 짝 맞추기](#)
 - [addRow 한 줄로 끝](#)
 - [주요 메서드](#)
- [레이아웃 중첩하기](#)
 - [레이아웃 안에 레이아웃 넣기](#)

레이아웃이란

좌표 대신 규칙으로 배치

- 레이아웃은 위젯의 위치를 좌표로 못 박는 대신, "세로로 쌓아라", "가로로 늘어놓아라" 같은 규칙으로 배치를 맡기는 도구입니다.
- 창 크기가 바뀌면 레이아웃이 그 안의 위젯 크기와 간격을 자동으로 다시 계산합니다. 그래서 어떤 화면에서도 깔끔하게 정렬됩니다.

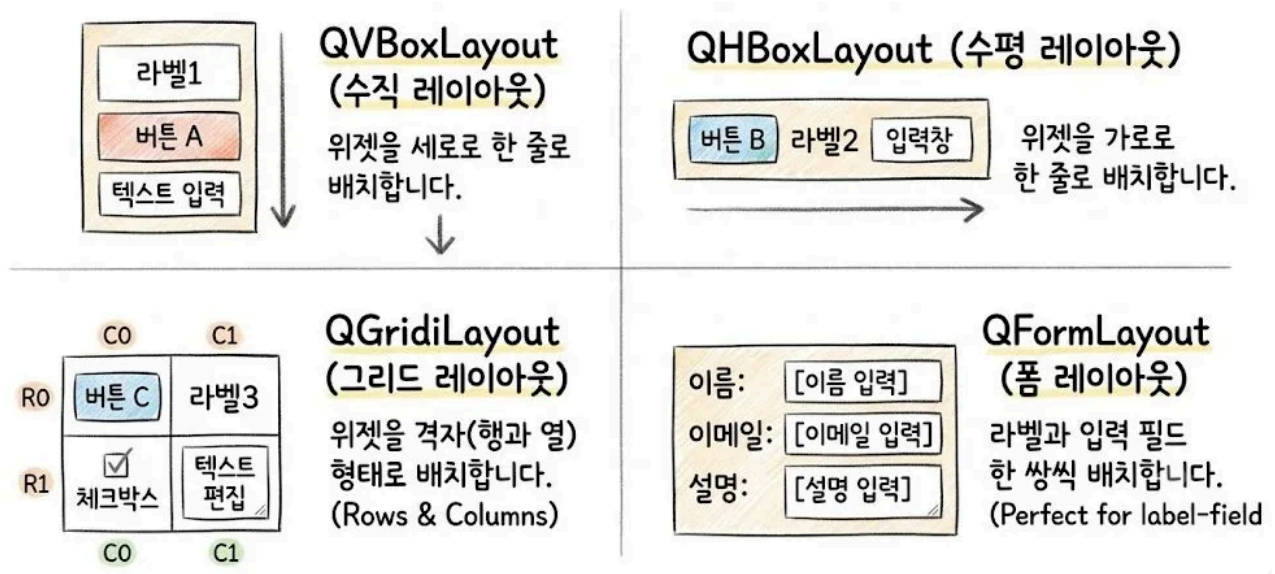
동일한 사용 순서

- 레이아웃 사용은 항상 ① 레이아웃 객체 생성 → ② 위젯 추가 → ③ 창에 레이아웃 지정 세 단계입니다.

```
layout = QVBoxLayout()
layout.addWidget(button)
window.setLayout(layout)
```

- 레이아웃을 쓰면 `move()` 나 `resize()` 로 위치·크기를 직접 지정할 필요가 없습니다.

네 가지 주요 레이아웃



QVBoxLayout

위에서 아래로 쌓기

- `QVBoxLayout` 은 추가한 위젯을 위에서 아래로 세로로 쌓는 레이아웃입니다. (V는 Vertical, 수직)
- `addWidget()` 을 호출한 순서대로 위에서부터 차례로 배치됩니다.

```

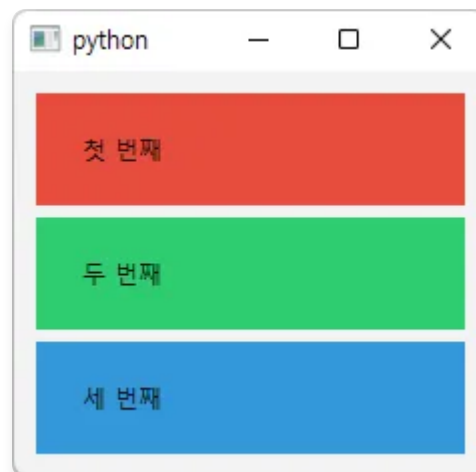
import sys
from PySide6.QtWidgets import QApplication, QWidget, QLabel, QVBoxLayout

app = QApplication(sys.argv)
window = QWidget()
layout = QVBoxLayout()

layout.addWidget(QLabel("첫 번째", styleSheet="padding:20px; background:#e74c3c"))
layout.addWidget(QLabel("두 번째", styleSheet="padding:20px; background:#2ecc71"))
layout.addWidget(QLabel("세 번째", styleSheet="padding:20px; background:#3498db"))

window.setLayout(layout)
window.show()
sys.exit(app.exec())

```



주요 메서드

메서드	설명
<code>addWidget(위젯)</code>	위젯을 아래쪽에 추가
<code>addLayout(레이아웃)</code>	다른 레이아웃을 중첩하여 추가
<code>addStretch()</code>	빈 공간(스프링)을 넣어 위젯을 한쪽으로 밀기
<code>addSpacing(int)</code>	지정한 픽셀만큼 고정 간격 추가
<code>setSpacing(int)</code>	위젯 사이의 간격 설정
<code>setContentsMargins(좌, 상, 우, 하)</code>	레이아웃 가장자리 여백 설정

QHBoxLayout

왼쪽에서 오른쪽으로 늘어놓기

- `QHBoxLayout` 은 위젯을 왼쪽에서 오른쪽으로 가로로 나열하는 레이아웃입니다. (H는 Horizontal, 수평)
- 사용법은 `QVBoxLayout` 과 완전히 같고, 배치 방향만 다릅니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QPushButton, QHBoxLayout

app = QApplication(sys.argv)
window = QWidget()
layout = QHBoxLayout()

layout.addWidget(QPushButton("이전", styleSheet="background:#95a5a6; padding:15px"))
layout.addWidget(QPushButton("확인", styleSheet="background:#3498db; padding:15px"))
layout.addWidget(QPushButton("다음", styleSheet="background:#95a5a6; padding:15px"))

window.setLayout(layout)
window.show()
sys.exit(app.exec())
```



스프링으로 정렬하기

- `addStretch()` 는 늘어나는 빈 공간을 넣어 줍니다. 위젯을 한쪽으로 밀거나 가운데로 모을 때 유용합니다.
- 예를 들어 버튼을 오른쪽 끝으로 보내려면, 버튼 앞에 스프링을 넣습니다.

```
layout = QHBoxLayout()

layout.addStretch()
layout.addWidget(QPushButton("이전", styleSheet="background:#95a5a6; padding:15px"))
```

QGridLayout

행과 열의 표처럼 배치

- `QGridLayout` 은 위젯을 행(row)과 열(column)로 이뤄진 격자에 배치하는 레이아웃입니다.
- `addWidget(위젯, 행, 열)` 형태로, 위젯을 놓을 칸의 위치를 행·열 번호로 지정합니다. 번호는 0 부터 시작합니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QLabel, QGridLayout

app = QApplication(sys.argv)
window = QWidget()
layout = QGridLayout()

style = "color:white; padding:20px; font-size:16px; background:"
layout.addWidget(QLabel("0,0", styleSheet=style + "#e74c3c"), 0, 0)
layout.addWidget(QLabel("0,1", styleSheet=style + "#2ecc71"), 0, 1)
layout.addWidget(QLabel("1,0", styleSheet=style + "#3498db"), 1, 0)
layout.addWidget(QLabel("1,1", styleSheet=style + "#f39c12"), 1, 1)

window.setLayout(layout)
window.show()
sys.exit(app.exec())
```



여러 칸 합치기

- 위젯이 여러 칸을 차지하게 하려면 `addWidget(위젯, 행, 열, 행개수, 열개수)` 로 차지할 행·열 수를 함께 지정합니다.
- 위 코드의 `QLabel("0,0")` 라인과 `QLabel("0,1")` 라인을 다음으로 대체합니다.

```
layout.addWidget(QLabel("0,0~1", styleSheet=style + "#e74c3c"), 0, 0, 1, 2)
```

주요 메서드

메서드	설명
<code>addWidget(위젯, 행, 열)</code>	지정한 칸에 위젯 배치
<code>addWidget(위젯, 행, 열, 행수, 열수)</code>	여러 칸에 걸쳐 위젯 배치
<code>setSpacing(int)</code>	칸 사이 간격 설정
<code>setRowStretch(행, 비율)</code>	특정 행이 늘어나는 비율 지정
<code>setColumnStretch(열, 비율)</code>	특정 열이 늘어나는 비율 지정

QFormLayout

라벨과 입력칸의 짝 맞추기

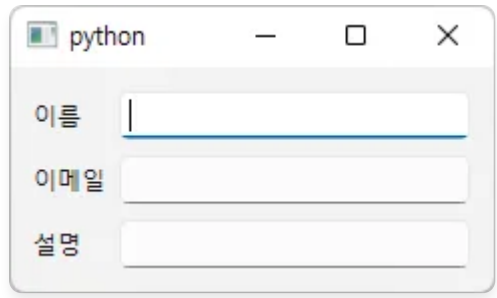
- `QFormLayout` 은 "이름표(라벨) + 입력칸" 한 쌍을 한 줄에 나란히 놓는 레이아웃입니다.
- 회원가입·설정 화면처럼 항목명과 입력 필드가 줄지어 나오는 양식(form)에 최적화되어 있습니다.

```
import sys
from PySide6.QtWidgets import (
    QApplication, QWidget, QLineEdit, QFormLayout
)

app = QApplication(sys.argv)
window = QWidget()
layout = QFormLayout()

layout.addRow("이름", QLineEdit())
layout.addRow("이메일", QLineEdit())
layout.addRow("설명", QLineEdit())

window.setLayout(layout)
window.show()
sys.exit(app.exec())
```



addRow 한 줄로 끝

- `addRow("라벨", 위젯)` 을 호출하면 왼쪽에 라벨, 오른쪽에 입력 위젯이 자동으로 짝지어 배치됩니다.
- 라벨을 직접 만들 필요 없이 문자열만 넘기면 되므로, 다른 레이아웃보다 양식 작성이 간결합니다.

주요 메서드

메서드	설명
<code>addRow(str, 위젯)</code>	라벨 문자열과 입력 위젯을 한 줄로 추가
<code>addRow(위젯, 위젯)</code>	라벨 위젯과 입력 위젯을 한 줄로 추가
<code>addRow(위젯)</code>	한 줄 전체를 차지하는 위젯 추가
<code>setSpacing(int)</code>	줄 사이 간격 설정

레이아웃 중첩하기

레이아웃 안에 레이아웃 넣기

- 실제 화면은 하나의 레이아웃만으로 만들기 어렵습니다. 레이아웃은 `addLayout()` 으로 다른 레이아웃을 자식으로 품을 수 있습니다.
- 예를 들어 전체를 세로(`QVBoxLayout`)로 쌓되, 가운데 라벨은 가로(`QHBoxLayout`)로 두는 식으로 조합합니다.

```

from PySide6.QtWidgets import QApplication, QWidget, QVBoxLayout, QHBoxLayout, QLabel

app = QApplication([])
window = QWidget()

main = QVBoxLayout(window)

top = QLabel("상 단")
top.setStyleSheet("background:#e74c3c; padding:15px;")
main.addWidget(top)

middle = QHBoxLayout()

left = QLabel("좌 ")
left.setStyleSheet("background:#3498db; padding:15px;")
right = QLabel("우 ")
right.setStyleSheet("background:#f39c12; padding:15px;")

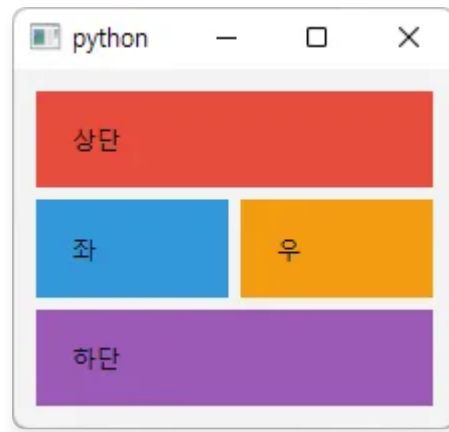
middle.addWidget(left)
middle.addWidget(right)
main.addLayout(middle)

bottom = QLabel("하 단")
bottom.setStyleSheet("background:#9b59b6; padding:15px;")
main.addWidget(bottom)

window.show()
app.exec()

```

이렇게 세로·가로 레이아웃을 겹겹이 조합하면, 좌표를 한 번도 계산하지 않고도 복잡한 화면을 자유롭게 구성할 수 있습니다.



시그널과 슬롯

GUI에서 "버튼을 누르면 무언가 실행되게" 하는 것이 바로 시그널(Signal)과 슬롯(Slot)입니다.

시그널은 버튼 클릭이나 값 변경 같은 사건이 일어났을 때 객체가 보내는 신호이며, 슬롯은 그 신호를 받아 실행되는 함수입니다. 둘을 연결(connect)해두면 시그널이 발생할 때마다 슬롯이 자동으로 호출됩니다.

이 강좌에서는 시그널과 슬롯의 기본 개념부터 연결 방법, 인자를 전달하는 처리, 그리고 emit으로 직접 시그널을 만들어 내보내는 방법까지 차근차근 다룹니다. 이 개념을 익히면 사용자 행동에 반응하는 인터페이스를 자유롭게 설계할 수 있습니다.

- [시그널과 슬롯이란](#)
 - [사건과 반응의 연결](#)
 - [연결하는 형태](#)
- [시그널-슬롯 연결하기](#)
 - [버튼 클릭에 반응하기](#)
 - [위젯 간 직접 연결](#)
- [위젯마다 다양한 시그널](#)
 - [위젯별 고유 시그널](#)
 - [주요 시그널](#)
 - [값 변화에 대한 반응](#)
- [시그널이 전달하는 값 받기](#)
 - [값의 전달](#)
 - [값의 선택적 수신](#)
- [람다로 인자 전달하기](#)
 - [사용자 지정 인자 전달](#)

- [직접 시그널 만들기](#)
 - [내장 시그널과 사용자 정의 시그널](#)
 - [사용자 정의 시그널 3단계](#)
 - [값을 담아 emit하기](#)
 - [여러 값을 전달하는 시그널](#)

시그널과 슬롯이란

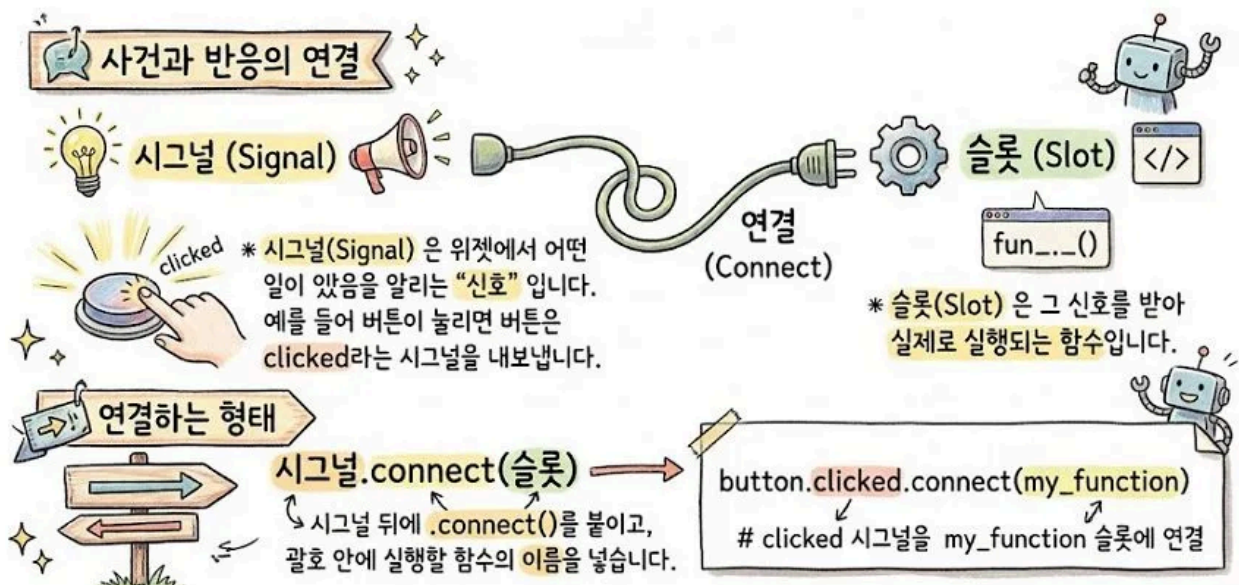
사건과 반응의 연결

- **시그널(Signal)** 은 위젯에서 어떤 일이 일어났음을 알리는 "신호" 입니다. 예를 들어 버튼이 눌리면 버튼은 `clicked` 라는 시그널을 내보냅니다.
- **슬롯(Slot)** 은 그 신호를 받아 실제로 실행되는 함수입니다.
- 이 둘을 **연결(connect)** 해 두면, 시그널이 발생할 때마다 연결된 슬롯이 자동으로 호출됩니다.

연결하는 형태

- 연결은 항상 **시그널 .connect(슬롯)** 형태로 작성합니다. 시그널 뒤에 `.connect()` 를 붙이고, 괄호 안에 실행할 함수의 이름을 넣습니다.

```
button.clicked.connect(my_function) # clicked 시그널을 my_function 슬롯에 연결
```



시그널-슬롯 연결하기

버튼 클릭에 반응하기

- 가장 흔한 예는 버튼의 `clicked` 시그널입니다. 버튼을 누르면 미리 연결해 둔 슬롯 함수가 실행 됩니다.
- 버튼을 누를 때마다 터미널에 메시지가 출력되는 간단한 예제를 앞서 위젯 관련 '라벨, 버튼, 입력 창'에서 살펴 보았습니다.

위젯 간 직접 연결

- 슬롯이 꼭 직접 만든 함수일 필요는 없습니다. 다른 위젯의 메서드도 슬롯이 될 수 있습니다.
- 아래는 버튼을 누르면 창이 닫히도록, `clicked` 시그널을 창의 `close` 메서드에 바로 연결한 예입니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QVBoxLayout, QPushButton

app = QApplication(sys.argv)
window = QWidget()

layout = QVBoxLayout(window)
button = QPushButton("닫기")

button.clicked.connect(window.close)

layout.addWidget(button)
window.show()

sys.exit(app.exec())
```

이처럼 시그널은 함수든 메서드든 호출 가능한 것이라면 무엇이든 슬롯으로 연결할 수 있습니다.



위젯마다 다양한 시그널

위젯별 고유 시그널

- `clicked` 는 버튼의 시그널일 뿐, 위젯마다 의미 있는 사건에 맞는 시그널이 따로 준비되어 있습니다.

주요 시그널

- 아래는 주요 위젯의 자주 쓰는 시그널입니다.
- 앞서 살펴본 위젯별 주요 시그널을 응용하면, 한층 더 상세한 동작을 설정할 수 있습니다.

위젯	시그널	발생 시점
<code>QPushButton</code>	<code>clicked</code>	버튼을 클릭했을 때
<code>QLineEdit</code>	<code>textChanged</code>	입력 글자가 바뀔 때
<code>QLineEdit</code>	<code>returnPressed</code>	입력창에서 Enter를 눌렀을 때
<code>QCheckBox</code>	<code>stateChanged</code>	체크 상태가 바뀔 때
<code>QComboBox</code>	<code>currentIndexChanged</code>	선택 항목이 바뀔 때
<code>QSlider</code>	<code>valueChanged</code>	슬라이더 값이 바뀔 때
<code>QSpinBox</code>	<code>valueChanged</code>	숫자 값이 바뀔 때

값 변화에 대한 반응

- 입력창의 `textChanged` 시그널을 라벨에 연결하면, 타이핑하는 즉시 라벨이 따라 바뀌는 화면을 만들 수 있습니다.

```

import sys
from PySide6.QtWidgets import (
    QApplication, QWidget, QLabel, QLineEdit, QVBoxLayout
)

app = QApplication(sys.argv)
window = QWidget()

edit = QLineEdit()
label = QLabel("여기에 표시됩니다.")

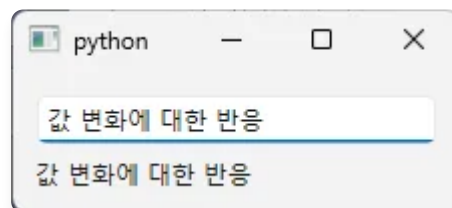
edit.textChanged.connect(label.setText)

layout = QVBoxLayout()
layout.addWidget(edit)
layout.addWidget(label)
window.setLayout(layout)

window.show()
sys.exit(app.exec())

```

`textChanged` 시그널은 바뀐 글자를 함께 넘겨주고, `setText` 는 그 글자를 받아 라벨에 표시합니다. 시그널이 값을 전달한다는 점이 다음 절의 핵심으로 이어집니다.



시그널이 전달하는 값 받기

값의 전달

- 어떤 시그널은 단순히 "사건이 일어났다"만 알리지만, 어떤 시그널은 **관련된 값을 함께** 전달합니다.

- 슬라이더를 움직이면 `valueChanged` 가 현재 값을 `setValue` 에 넘기고, 프로그레스바가 그 값을 그대로 표시합니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QVBoxLayout, QSlider, QProgressBar
from PySide6.QtCore import Qt

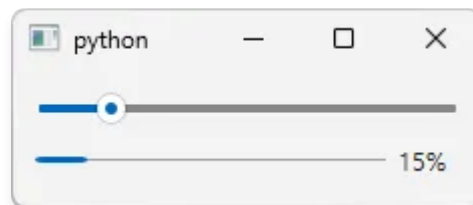
app = QApplication(sys.argv)
window = QWidget()
layout = QVBoxLayout(window)

slider = QSlider(Qt.Horizontal)
progress_bar = QProgressBar()

slider.valueChanged.connect(progress_bar.setValue)

layout.addWidget(slider)
layout.addWidget(progress_bar)
window.show()

sys.exit(app.exec())
```



값의 선택적 수신

- 시그널이 값을 보내더라도, 슬롯에서 그 값이 필요 없다면 매개변수 없이 받아도 됩니다. 버튼의 `clicked` 시그널이 대표적입니다.
- 다만 슬롯이 받는 매개변수의 개수는 시그널이 보내는 값의 개수보다 많을 수 없습니다. 필요한 만큼만 앞에서부터 받는다고 생각하면 됩니다.

람다로 인자 전달하기

사용자 지정 인자 전달

- 시그널이 보내주는 값 말고, **내가 정한 값**을 슬롯에 넘기고 싶을 때가 있습니다. 예를 들어 버튼이 여러 개인데 어느 버튼이 눌렸는지 슬롯에서 구분하고 싶은 경우입니다.
- 이때 **람다(lambda)**를 사용합니다. 람다는 그 자리에서 만드는 이름 없는 함수로, 슬롯을 감싸 원하는 인자를 함께 넘길 수 있습니다.

```
import sys
from PySide6.QtWidgets import (
    QApplication, QWidget, QLabel, QPushButton, QVBoxLayout
)

def on_click(what):
    label.setText(f"{what} 버튼을 눌렀습니다.")

app = QApplication(sys.argv)
window = QWidget()

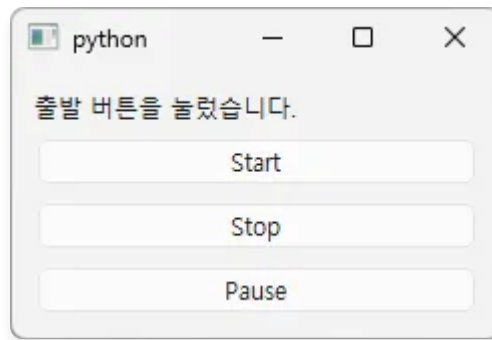
label = QLabel("버튼을 눌러보세요.")
layout = QVBoxLayout()
layout.addWidget(label)

buttons = [("Start", "출발"), ("Stop", "정지"), ("Pause", "일시정지")]

for text, message in buttons:
    button = QPushButton(text)
    button.clicked.connect(lambda checked=False, message=message: on_click(message))
    layout.addWidget(button)

window.setLayout(layout)
window.show()
sys.exit(app.exec())
```

람다 덕분에 같은 슬롯 함수를 여러 버튼이 서로 다른 값으로 재사용할 수 있습니다.



직접 시그널 만들기

내장 시그널과 사용자 정의 시그널

- `clicked` 처럼 Qt가 알아서 발생시키는 내장 시그널과 달리, "데이터 로딩 완료", "점수 변경" 등 내 프로그램만의 특별한 사건은 직접 시그널을 만들어야 합니다.
- 이를 사용자 정의 시그널이라 하며, Qt가 발생 시점을 알 수 없으므로 원하는 순간에 직접 `emit()` 을 호출해 신호를 보내야 합니다.

사용자 정의 시그널 3단계

- 시그널은 반드시 `QObject` 를 상속받은 클래스 내에서 사용해야 하며, 다음 세 단계를 거칩니다.
 - 1. 정의: `Signal` 을 클래스 속성으로 선언 (전달할 값의 자료형 지정)
 - 2. 연결: `connect` 를 사용해 실행할 슬롯에 연결
 - 3. 발생: 사건이 일어난 순간 `시그널.emit(값)` 으로 신호 발송
- 기본 사용 법

```

시그널이름 = Signal(int)      # 1. 정의 (int 값을 전달하겠다고 선언)
...
self.시그널이름.connect(슬롯) # 2. 연결 (신호가 오면 슬롯 실행)
...
self.시그널이름.emit(현재값)  # 3. 발생 (원하는 순간에 값을 담아 발송)

```

`connect` 가 "신호가 오면 실행하라"는 예약이라면, `emit` 은 "지금 신호를 보낸다"는 실행입니다.

값을 담아 emit하기

- 버튼을 누르면 숫자가 증가하고, 변경된 값을 직접 만든 `countChanged` 시그널로 라벨에 전달하는 예제입니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QLabel, QPushButton, QVBoxLayout
from PySide6.QtCore import Signal

class Counter(QWidget):
    countChanged = Signal(int)          # 1. 정의: 정수로 시그널 정의

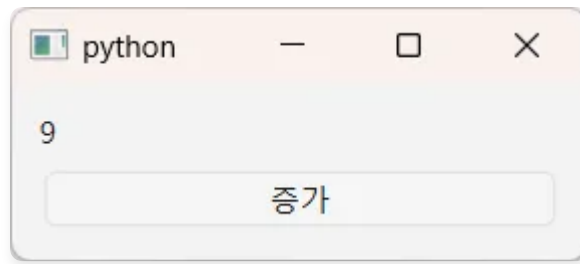
    def __init__(self):
        super().__init__()
        self.count = 0
        label = QLabel("0")
        button = QPushButton("증가")
        button.clicked.connect(self.increase)
        self.countChanged.connect(label.setNum) # 2. 연결: countChanged 신호가 오면 라벨에

        layout = QVBoxLayout(self)
        layout.addWidget(label)
        layout.addWidget(button)

    def increase(self):
        self.count += 1
        self.countChanged.emit(self.count) # 3. 발생: 값이 바뀔 때마다 변경된 숫자를 담아

app = QApplication(sys.argv)
window = Counter()
window.show()
sys.exit(app.exec())
```

버튼 클릭(`clicked`) → `increase()` 실행 → 값 증가 및 `emit()` 발송 → 라벨 업데이트 (`setNum`)



여러 값을 전달하는 시그널

- `Signal(str, int)` 처럼 자료형을 여러 개 지정하면, `emit` 때 그 개수만큼 값을 함께 넘길 수 있습니다. 슬롯은 넘어온 값을 순서대로 받습니다.

```
statusChanged = Signal(str, int)  # 문자열과 정수를 함께 전달
...
self.statusChanged.emit("완료", 100)  # 두 값을 함께 emit
```

- 이렇게 만든 사용자 정의 시그널도 내장 시그널과 똑같이 `connect` 로 연결하고, 필요하면 람다로 인자를 가공해 넘길 수 있습니다. 직접 만든 시그널과 내장 시그널을 다루는 방식은 완전히 동일합니다.

메뉴, 툴바, 상태바

본격적인 데스크톱 애플리케이션은 화면 상단에 기능을 분류해 담은 **메뉴**가 있고, 그 아래에는 자주 쓰는 기능을 아이콘 버튼으로 모아둔 **툴바**가 있으며, 창 맨 아래에는 현재 상태나 안내 문구를 보여주는 **상태바**가 있습니다. 이 세 가지는 QMainWindow라는 특별한 창이 미리 마련해 둔 자리에 손쉽게 붙일 수 있습니다.

이 장에서는 QMainWindow의 구조를 이해하고, 메뉴·툴바·상태바를 하나씩 만들어 실제 프로그램다운 골격을 완성합니다.

- [QMainWindow의 구조](#)
 - [QWidget이 아닌 QMainWindow](#)
 - [영역 구성](#)
 - [기본 뼈대](#)
- [QAction 이해하기](#)
 - [하나의 동작, 여러 위치](#)
 - [QAction 만들기](#)
 - [주요 메서드](#)
- [메뉴](#)
 - [메뉴바와 메뉴](#)
- [툴바](#)
 - [툴바 추가](#)
 - [주요 메서드](#)
- [상태바](#)
 - [상태바 얻기와 메시지 표시](#)
 - [주요 메서드](#)
- [전체 예제](#)
 - [메뉴·툴바·상태바를 모두 갖춘 창](#)

QMainWindow의 구조

QWidget이 아닌 QMainWindow

- `QWidget` 은 아무 장식 없는 빈 창이라, 메뉴나 툴바를 붙일 수 없습니다.
- `QMainWindow` 는 메뉴바·툴바·상태바·중앙 위젯이 들어갈 자리를 **미리 구획해 둔** 창입니다.

영역 구성

- `QMainWindow` 는 위에서 아래로 다음 영역이 정해진 순서대로 배치됩니다.

영역	위치	담당
메뉴바(Menu Bar)	최상단	기능을 분류한 메뉴 모음
툴바(Tool Bar)	메뉴바 아래	자주 쓰는 기능의 아이콘 버튼
중앙 위젯(Central Widget)	가운데	실제 작업 화면 (반드시 하나 지정)
상태바(Status Bar)	최하단	상태 메시지·안내 표시

기본 뼈대

- 짧은 코드에서는 `QMainWindow` 객체를 직접 만들어 각 영역을 구성하는 방식이 간단합니다.

```
import sys
from PySide6.QtWidgets import QApplication, QMainWindow, QTextEdit

app = QApplication(sys.argv)

window = QMainWindow()

editor = QTextEdit()
window.setCentralWidget(editor)

window.show()
sys.exit(app.exec())
```

`setCentralWidget()` 으로 가운데에 들어갈 핵심 위젯을 지정합니다. 이 자리는 비워둘 수 없습니다.

QAction 이해하기

- 메뉴바, 툴바, 상태바를 본격적으로 만들기 전에 짚고 넘어가야 할 핵심 개념이 있습니다. 바로 `QAction` 입니다.

하나의 동작, 여러 위치

- 메뉴와 툴바는 종종 '열기', '저장'처럼 같은 기능을 공유합니다.
- `QAction` 은 이름, 아이콘, 단축키, 실행 함수를 한곳에 담아 하나의 동작을 정의합니다.
- 동일한 `QAction` 을 메뉴와 툴바에 등록하면 동작과 활성화 상태가 자동으로 동기화됩니다.

QAction 만들기

- 텍스트(표시 이름)와 부모 창을 넘겨 생성하고, 옵션을 설정한 뒤 `triggered` 시그널에 실행할 함수를 연결합니다.

```
from PySide6.QtGui import QAction

open_action = QAction("열기", window)
open_action.triggered.connect(some_func)
```

주요 메서드

메서드	설명
<code>setShortcut(seq)</code>	단축키 지정 (예: <code>Ctrl+O</code>)
<code>setStatusTip(text)</code>	마우스를 올렸을 때 상태바에 표시할 안내 문구
<code>setIcon(icon)</code>	툴바 등에 표시할 아이콘 지정
<code>setCheckable(True)</code>	켜고 끌 수 있는 토글형 동작으로 설정

- `setShortcut("Ctrl+O")` 처럼 문자열로도 단축키를 지정할 수 있고, 정밀하게 다루려면 `QKeySequence` 를 넘겨도 됩니다.

메뉴

메뉴바와 메뉴

- `QMainWindow` 의 `menuBar()` 로 메뉴바를 얻고, `addMenu()` 로 '파일', '편집' 같은 상위 메뉴를 추가합니다.
- 각 메뉴에는 `QAction` 을 `addAction()` 으로 넣습니다.

```
import sys
from PySide6.QtWidgets import QApplication, QMainWindow
from PySide6.QtGui import QAction

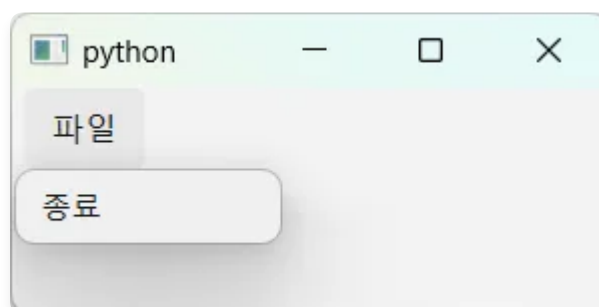
app = QApplication(sys.argv)

window = QMainWindow()

exit_action = QAction("종료", window)
exit_action.triggered.connect(app.quit)

file_menu = window.menuBar().addMenu("파일")
file_menu.addAction(exit_action)

window.show()
sys.exit(app.exec())
```



툴바

툴바 추가

- `addToolBar()` 로 툴바를 만듭니다.

```
import sys
from PySide6.QtWidgets import QApplication, QMainWindow
from PySide6.QtGui import QAction

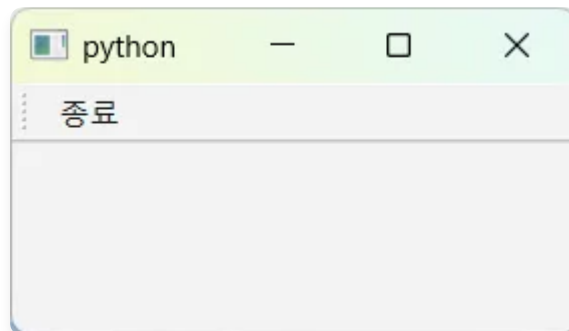
app = QApplication(sys.argv)

window = QMainWindow()

exit_action = QAction("종료", window)
exit_action.triggered.connect(app.quit)

toolbar = window.addToolBar("기 본 툴바")
toolbar.addAction(exit_action)

window.show()
sys.exit(app.exec())
```



주요 메서드

메서드	설명
<code>addAction(action)</code>	동작(아이콘 버튼) 추가
<code>addSeparator()</code>	버튼 사이 구분선 추가
<code>addWidget(widget)</code>	툴바에 일반 위젯(콤보박스 등) 삽입
<code>setMovable(bool)</code>	사용자가 투바를 끌어서 옮길 수 있는지 설정
<code>setToolButtonStyle(style)</code>	아이콘/글자 표시 방식 설정

상태바

상태바 얻기와 메시지 표시

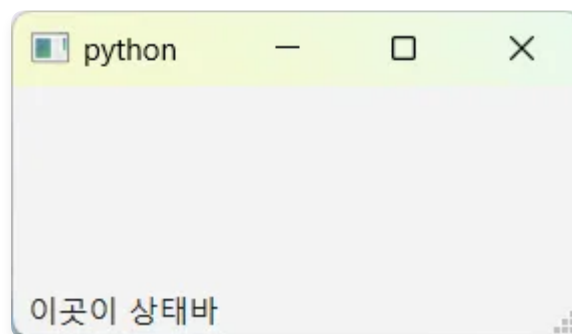
- `statusBar()` 를 호출하면 창 하단의 상태바를 얻습니다.

```
import sys
from PySide6.QtWidgets import QApplication, QMainWindow

app = QApplication(sys.argv)

window = QMainWindow()
window.statusBar().showMessage("이곳이 상태바")

window.show()
sys.exit(app.exec())
```



주요 메서드

메서드	설명
<code>showMessage(text, msec=0)</code>	임시 메시지 표시 (<code>msec</code> 만큼 후 사라짐, 0이면 계속)
<code>clearMessage()</code>	임시 메시지 지우기
<code>addWidget(widget)</code>	상태바 왼쪽에 위젯 추가
<code>addPermanentWidget(widget)</code>	상태바 오른쪽에 고정 위젯 추가

- `QAction` 에 `setStatusTip()` 을 설정해 두면, 마우스를 메뉴·툴바 항목에 올렸을 때 그 문구가 상태바에 자동으로 나타납니다.

전체 예제

메뉴·툴바·상태바를 모두 갖춘 창

- 지금까지의 내용을 하나로 합친, 간단한 텍스트 편집 창입니다.
- `open_action` 과 `exit_action` 을 만들어 메뉴와 툴바에 **같은 객체**로 추가하고, 상태바에는 시작 메시지를 띄웁니다.

```
import sys
from PySide6.QtWidgets import QApplication, QMainWindow, QTextEdit
from PySide6.QtGui import QAction

app = QApplication(sys.argv)

window = QMainWindow()
window.resize(400, 300)

editor = QTextEdit()
window.setCentralWidget(editor)

open_action = QAction("열기", window)
open_action.setShortcut("Ctrl+O")
open_action.triggered.connect(lambda: window.statusBar().showMessage("열기 실행됨!", 3000))

exit_action = QAction("종료", window)
exit_action.triggered.connect(app.quit)

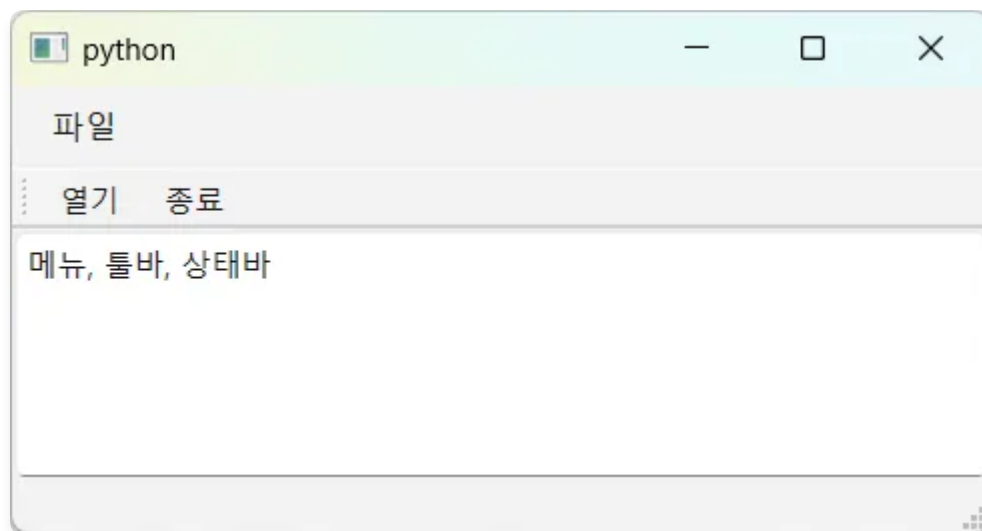
file_menu = window.menuBar().addMenu("파일")
file_menu.addAction(open_action)
file_menu.addSeparator()
file_menu.addAction(exit_action)

toolbar = window.addToolBar("기본 툴바")
toolbar.addAction(open_action)
toolbar.addAction(exit_action)

window.statusBar().showMessage("이곳이 상태바")

window.show()
sys.exit(app.exec())
```

- 창을 띄우면 상단에 '파일' 메뉴와 '기본 툴바'가, 하단 상태바에 "이곳이 상태바"가 표시됩니다.
- **Ctrl+O** 를 누르거나 메뉴·툴바의 '열기'를 선택하면 상태바에 "열기 실행됨!"이 3초 동안 잠시 나타납니다.
- '종료'를 선택하면 프로그램이 끝납니다.
- 메뉴와 툴바의 '열기'·'종료'는 각각 같은 **QAction** 이므로 어느 쪽을 눌러도 완전히 동일하게 작동합니다.



대화상자 (QDialog)

이 장에서는 표준 대화상자(QMessageBox, QFileDialog, QInputDialog, QColorDialog, QFontDialog)와, 직접 만들어 쓰는 사용자 정의 대화상자(QDialog)를 익힙니다.

- [메시지 박스](#)
 - [가장 많이 쓰는 알림 창](#)
 - [종류별 정적 메서드](#)
 - [사용자의 응답 받기](#)
- [파일 대화상자](#)
 - [파일·폴더를 고르게 하기](#)
 - [열기·저장·폴더 선택](#)
 - [주요 메서드](#)
 - [필터 문법](#)
- [입력 대화상자](#)
 - [값 하나를 간단히 입력받기](#)
 - [주요 메서드](#)
- [색상·폰트 대화상자](#)
 - [색상 선택](#)
 - [폰트 선택](#)
- [사용자 정의 대화상자](#)
 - [직접 만드는 대화상자](#)
 - [exec\(\)로 띄우고 결과 받기](#)
 - [주요 메서드](#)

메시지 박스

가장 많이 쓰는 알림 창

- `QMessageBox` 는 짧은 메시지를 보여주고 버튼으로 응답을 받는, 가장 흔한 대화상자입니다.

- 정보 알림, 경고, 오류, 질문(예/아니오) 등 상황에 맞는 정적 메서드를 제공합니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget, QPushButton, QMessageBox

app = QApplication(sys.argv)
window = QWidget()

def show_message():
    QMessageBox.warning(window, "경고", "이 작업은 CPU 사용률이 매우 높습니다.")

button = QPushButton("주의", window)
button.clicked.connect(show_message)
button.move(80, 80)

window.show()
sys.exit(app.exec())
```

종류별 정적 메서드

메서드	설명
<code>QMessageBox.information(...)</code>	일반 정보 알림
<code>QMessageBox.warning(...)</code>	경고 표시
<code>QMessageBox.critical(...)</code>	심각한 오류 표시
<code>QMessageBox.question(...)</code>	예/아니오 등 질문
<code>QMessageBox.about(...)</code>	프로그램 정보 표시

사용자의 응답 받기

- `question()` 은 사용자가 누른 버튼을 **반환값**으로 돌려줍니다. 이 값을 비교해 분기합니다.
- 버튼 종류는 `QMessageBox.StandardButton` 으로 지정하며, `|` (OR)로 여러 개를 함께 표시할 수 있습니다.

```

reply = QMessageBox.question(
    window, "확인",
    "정말 삭제하시겠습니까?",
    QMessageBox.Yes | QMessageBox.No
)

if reply == QMessageBox.Yes:
    print("삭제 진행")
else:
    print("취소됨")

```

주요 버튼 상수	의미
<code>QMessageBox.Ok</code>	확인
<code>QMessageBox.Cancel</code>	취소
<code>QMessageBox.Yes</code>	예
<code>QMessageBox.No</code>	아니오
<code>QMessageBox.Save</code>	저장
<code>QMessageBox.Discard</code>	버림

파일 대화상자

파일·폴더를 고르게 하기

- `QFileDialog` 는 사용자가 **파일**이나 **폴더**를 직접 선택하게 하는 대화상자입니다.
- 선택 결과를 **경로 문자열**로 반환하므로, 그 경로로 파일을 열거나 저장하면 됩니다.

열기·저장·폴더 선택

- 정적 메서드는 (선택된 경로, 필터 문자열)을 **튜플**로 반환합니다. 보통 첫 번째 값만 사용합니다.
- 사용자가 취소하면 경로 자리에 **빈 문자열**이 돌아옵니다.

```

import sys
from PySide6.QtWidgets import QApplication, QWidget, QVBoxLayout, QPushButton, QFileDialog

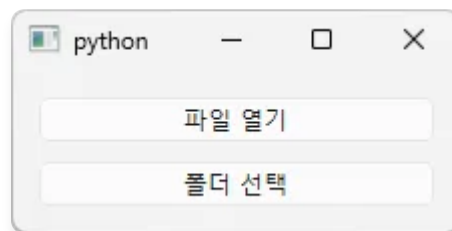
app = QApplication(sys.argv)
window = QWidget()
layout = QVBoxLayout(window)

btn_file = QPushButton("파일 열기")
btn_folder = QPushButton("폴더 선택")
layout.addWidget(btn_file)
layout.addWidget(btn_folder)

btn_file.clicked.connect(lambda: print("선택한 파일:", QFileDialog.getOpenFileName(window,
btn_folder.clicked.connect(lambda: print("선택한 폴더:", QFileDialog.getExistingDirectory(

window.show()
sys.exit(app.exec())

```



주요 메서드

메서드	반환값	설명
<code>getOpenFileName(...)</code>	(경로, 필터)	파일 1개를 열기 위해 선택
<code>getOpenFileNames(...)</code>	(경로 리스트, 필터)	파일 여러 개를 선택
<code>getSaveFileName(...)</code>	(경로, 필터)	저장할 경로를 지정
<code>getExistingDirectory(...)</code>	경로 문자열	폴더를 선택

필터 문법

- 필터는 `"설명 (*.확장자)"` 형식이며, 여러 개는 `;;` (세미콜론 두 개)로 구분합니다.

- 예: "이미지 (*.png *.jpg);;텍스트 (*.txt);;모든 파일 (*)"

입력 대화상자

값 하나를 간단히 입력받기

- `QInputDialog` 는 문자열, 정수, 실수, 항목 선택 등 간단한 값 하나를 입력받는 대화상자입니다.
- 정적 메서드는 (입력값, 확인 여부)를 튜플로 반환합니다. 두 번째 값이 `True` 일 때만 확인을 누른 것입니다.

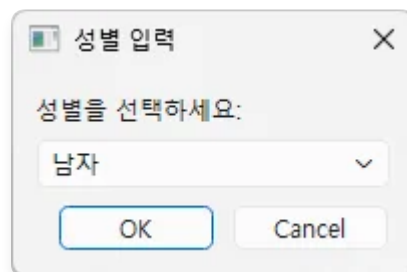
```
from PySide6.QtWidgets import QApplication, QWidget, QInputDialog

app = QApplication([])
window = QWidget()

text, ok = QInputDialog.getText(window, "이름 입력", "이름을 입력하세요:")
if ok and text:
    print("이름: ", text)

num, ok = QInputDialog.getInt(window, "나이 입력", "나이:", 20, 0, 120)
if ok:
    print("나이: ", num)

genders = ["남자", "여자"]
gender, ok = QInputDialog.getItem(window, "성별 입력", "성별을 선택하세요:", genders, 0, 1)
if ok and gender:
    print("성별: ", gender)
```



주요 메서드

메서드	설명
<code>getText(...)</code>	한 줄 문자열을 입력받음
<code>getMultiLineText(...)</code>	여러 줄 문자열을 입력받음
<code>getInt(...)</code>	정수를 입력받음
<code>getDouble(...)</code>	실수를 입력받음
<code>getItem(...)</code>	목록 중 하나를 선택받음

색상·폰트 대화상자

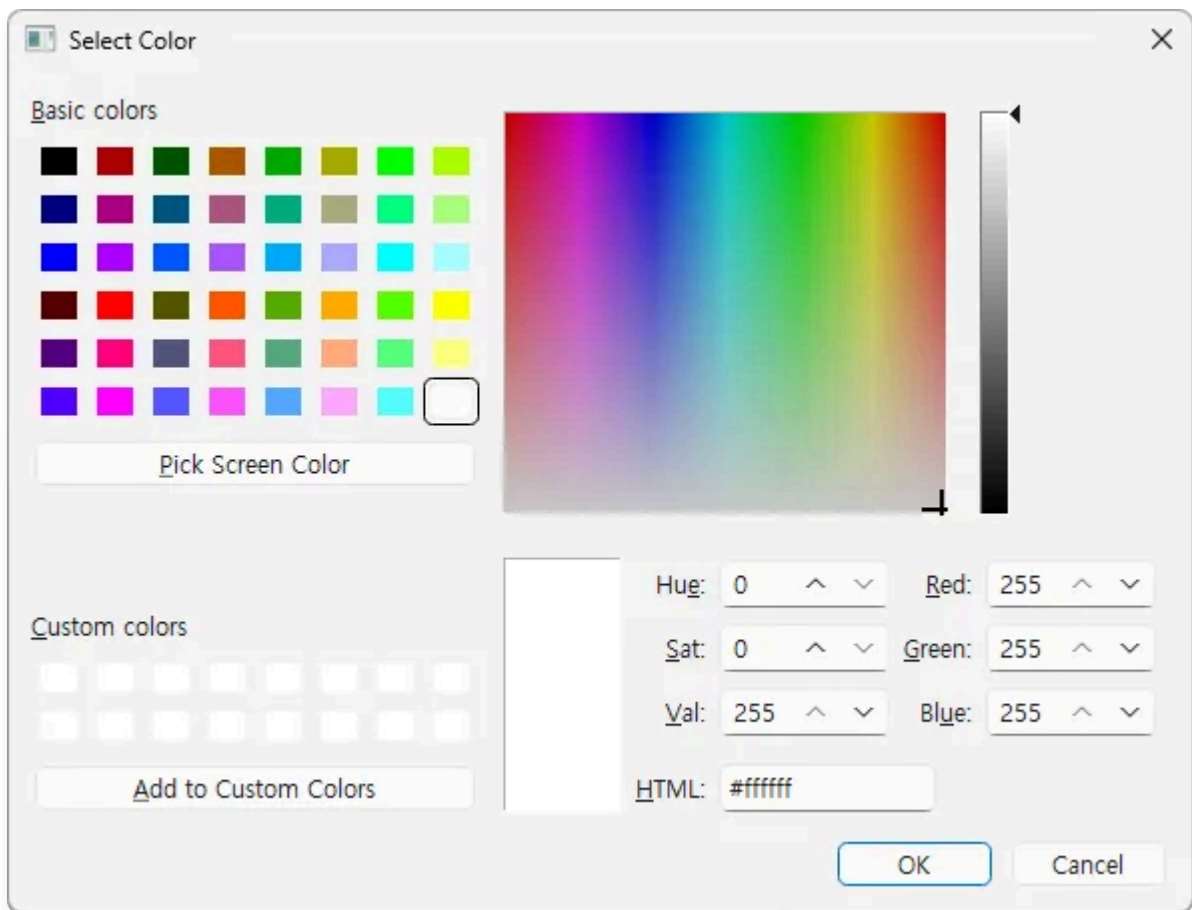
색상 선택

- `QColorDialog.getColor()` 는 색상 선택 창을 띄우고 선택한 색을 `QColor` 객체로 반환합니다.
- 반환된 색이 유효한지는 `isValid()` 로 확인합니다. 취소 시 유효하지 않은 색이 돌아옵니다.

```
import sys
from PySide6.QtWidgets import QApplication, QColorDialog

app = QApplication(sys.argv)

color = QColorDialog.getColor()
if color.isValid():
    print("선택한 색:", color.name())
```



폰트 선택

- `QFontDialog.getFont()` 는 폰트 선택 창을 띄우고 (확인 여부, 폰트)를 튜플로 반환합니다.
- 반환된 `QFont` 를 위젯에 `setFont()` 로 적용합니다.

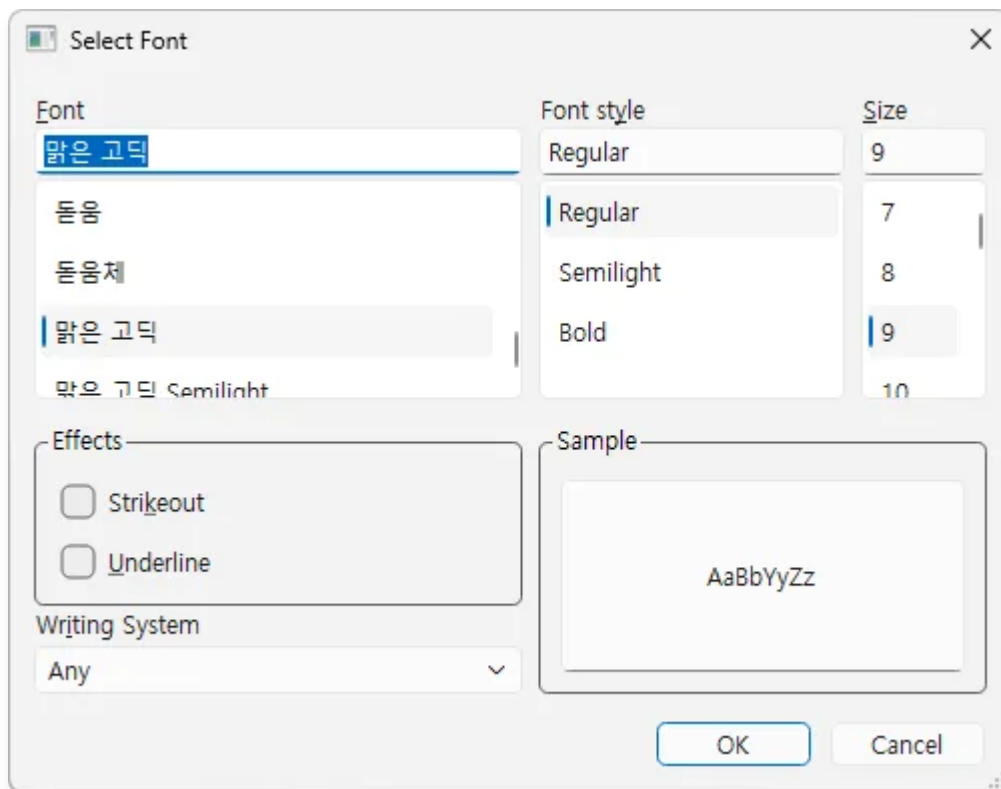
```
import sys
from PySide6.QtWidgets import QApplication, QLabel, QFontDialog

app = QApplication(sys.argv)

label = QLabel("폰트 선택 대화상자")

ok, font = QFontDialog.getFont()
if ok:
    label.setFont(font)
    print("선택한 폰트:", font.family(), font.pointSize())

label.show()
sys.exit(app.exec())
```



사용자 정의 대화상자

직접 만드는 대화상자

- 표준 대화상자로 부족할 때는 `QDialog` 를 상속해 원하는 위젯을 자유롭게 배치한 대화상자를 만듭니다.
- `QDialog` 는 일반 창과 달리 `accept()` (확인), `reject()` (취소) 같은 대화상자 전용 동작을 갖고 있습니다.

```

import sys
from PySide6.QtWidgets import (
    QApplication, QDialog, QVBoxLayout, QLabel,
    QLineEdit, QComboBox, QRadioButton, QButtonGroup, QDialogButtonBox
)

def create_profile_dialog(parent=None):
    dialog = QDialog(parent)
    dialog.setWindowTitle("사용자 프로필")
    layout = QVBoxLayout()

    layout.addWidget(QLabel("닉네임:"))
    nick = QLineEdit()
    layout.addWidget(nick)

    layout.addWidget(QLabel("직업:"))
    job = QComboBox()
    job.addItem("사무·관리직", "생산·현장직", "전문·연구직", "서비스 및 기타")
    layout.addWidget(job)

    layout.addWidget(QLabel("알림 수신 동의:"))
    yes = QRadioButton("동의")
    no = QRadioButton("거부")
    yes.setChecked(True)
    group = QButtonGroup(dialog)
    group.addButton(yes)
    group.addButton(no)
    layout.addWidget(yes)
    layout.addWidget(no)

    buttons = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
    buttons.accepted.connect(dialog.accept)
    buttons.rejected.connect(dialog.reject)
    layout.addWidget(buttons)

    dialog.setLayout(layout)
    return dialog, nick, job, yes

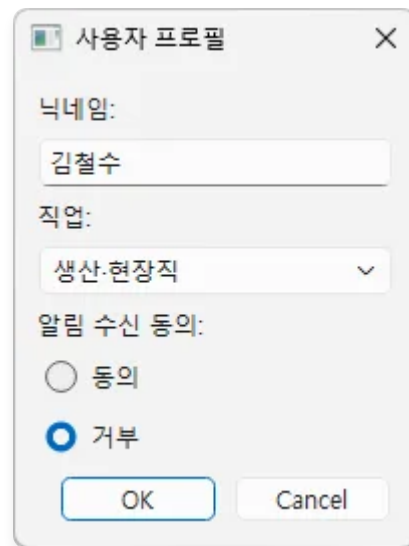
app = QApplication(sys.argv)
dialog, nick, job, yes = create_profile_dialog()

```

```

if dialog.exec() == QDialog.Accepted:
    print("닉네임:", nick.text())
    print("직업:", job.currentText())
    print("알림 수신:", "동의" if yes.isChecked() else "거부")
else:
    print("취소됨")

```



exec()로 띄우고 결과 받기

- 만든 대화상자는 `exec()` 로 모달 실행합니다. 사용자가 닫을 때까지 그 줄에서 대기합니다.
- `exec()` 는 확인을 누르면 `QDialog.Accepted` , 취소나 닫기면 `QDialog.Rejected` 를 반환합니다.

```

dialog, nick, job, yes = create_profile_dialog()
if dialog.exec() == QDialog.Accepted:
    print("닉네임:", nick.text())
    print("직업:", job.currentText())
    print("알림 수신:", "동의" if yes.isChecked() else "거부")
else:
    print("취소됨")

```

주요 메서드

메서드	설명
<code>exec()</code>	모달로 실행, 결과 코드 반환
<code>accept()</code>	확인 처리 후 닫기 (Accepted 반환)
<code>reject()</code>	취소 처리 후 닫기 (Rejected 반환)
<code>open()</code>	모달리스로 표시
<code>setModal(bool)</code>	모달 여부 설정
<code>result()</code>	마지막 실행 결과 코드 반환